AGENT EVALUATION
SCIENCE

When the Pipeline Is Wrong: Auditing the Evaluation of Scientific Agents

Sarvesh Tiku[1] Gauri Sharma[1]
[1] Georgia Institute of Technology
stiku6@gatech.edu gsharma80@gatech.edu

Abstract

A benchmark that credits a scientific agent's conclusion conflates three things: whether it is true, whether the agent's experiment supports it, and whether the grader checked. We separate them in hypothesis-testing tasks with known generating processes, using a frozen oracle that recomputes confirmatory evidence independently of the agent. Re-executing every agent-written confirmation script in a 1,680-run experiment (7 models) shows that 42% of single confirmation calls fit several candidate hypotheses on held-out data. Of 19 traces credited only by an answer-only grader ($+4.5$ pp), 8 contradict the agent's own significant test. The other 11 were rejected by the reference grader itself, 10 of them correct readings of a non-significant test whose output label broke format. Under a redesigned protocol where the harness runs one locked test and decisions cite an immutable record (640 runs, 40 new tasks), every confirmed decision matched its evidence. An answer-only grader still added $+3.1$ pp by crediting untested "no discovery" submissions, enough to move the lowest-ranked model to first. False discoveries concentrate on datasets where a deterministic baseline also errs. Scientific-agent benchmarks should generate evidence in the harness and score truth, evidence, and credit separately.

Keywords: Scientific agents; Agent evaluation; Evidential validity; Grader design; Fault injection; Reproducibility

1 Introduction

A scientific-agent benchmark that marks a conclusion "correct" makes three claims at once. The conclusion agrees with how the data were generated, the agent's experiment supports it, and the grader credited it for that reason. These are different measurement objects. A conclusion can be true but unsupported, as when an agent ignores a significant confirmatory result and the effect happens to be absent. It can also be supported but untrue, as when the confirmation sample contains a chance association. Benchmarks for data-driven discovery (Chen et al., 2025; Majumder et al., 2025; Wang et al., 2026; Lu et al., 2024) grade conclusions through pipelines that execute agent code, hold out data, and check reported results. We ask: *when truth, evidence, and credit disagree, which disagreements does a pipeline reward, and which pipeline faults create new ones?* Controlled hypothesis-testing tasks with known generating processes make all three quantities observable.

Our contributions are:

- a framework that scores truth, evidence, decision consistency, and credit separately, with a frozen oracle that recomputes confirmatory evidence independently of the agent;
- an instrumented audit of a 1,680-run experiment in which agents wrote their own confirmation scripts. 42% of single confirmation calls fit more than one candidate on held-out data, and 11 of the 19 submissions credited only by an answer-only grader were not evidence violations;

- a protocol in which the harness runs one locked test and decisions cite an immutable evidence record, with single-boundary faults logged for manifestation, evaluated on 40 new tasks.

2 Related Work

Kapoor et al. (2024) and Zhu et al. (2025) catalog validity flaws in agent benchmarks. Shao et al. (2026) define protocol validity as the intended capability remaining necessary for success, and audit shortcut exposure across 15 benchmarks. Chen (2026) treats an agent harness as software under test, using oracle checks, injected defects, and replay to correct labels. He et al. (2026) show that a fault-injection result supports a claim about an agent only if the faulty observation and the counter-evidence actually became visible, measured per execution. We therefore claim no novelty for fault injection, replay, or the observation that dropping a grading check inflates scores. We add the scientific-evidence dimension: with a known generating process and an independent oracle, *correct* and *supported* become separately measurable. Following He et al. (2026), we report whether each fault triggered and reached the agent. Among scientific-agent benchmarks (Chan et al., 2025; Starace et al., 2025; Siegel et al., 2024), FIRE-Bench (Wang et al., 2026) asks for evidence-backed conclusions. We ask whether the instrument can tell such conclusions from merely correct ones. Preregistration and holdout discipline (Nosek et al., 2018; Dwork et al., 2015) guard against analytic flexibility (Simmons et al., 2011; Gelman and Loken, 2014). Our protocol enforces them in the harness, consistent with treating validity as a property of the inferences drawn from a score (Messick, 1995).

3 Measurement Framework

Task family. Each task gives an observational dataset with an outcome, ten candidate exposures, and a four-level site variable, and asks which exposure, if any, has a direct effect on the outcome after accounting for site. Data come from a linear model (Appendix A.1) in which site shifts the outcome, one decoy exposure is confounded by site, and in *effect* tasks one exposure has a direct effect $\beta \in [0.15, 0.25]$ per exposure standard deviation, in units of the noise standard deviation. Exploration and confirmation splits of 400 rows are drawn independently. Site is stored as text, one exposure contains `"NA"` strings, and 1% of outcomes are missing. The four surface framings (agronomy, clinical, materials, ecology) share one statistical family. A Monte Carlo study of 10,000 dataset pairs (Appendix A.4) confirms that the fixed confirmatory test rejects at 5.0% under the null and that the decoy defeats unadjusted analyses in every simulated null task.

3.1 Four separately scored quantities

A task has hidden truth $Z \in \{\varnothing, x_1, \ldots, x_{10}\}$. The agent locks a hypothesis H, obtains evidence E, and submits $D \in \{\text{disc}(x),\ \text{ncd},\ \text{abstain}\}$, where ncd means "no confirmed discovery". Let p_H be the p-value of the frozen *oracle* test of H on the confirmation split: OLS of the outcome on H and site indicators, complete cases, classical standard errors, two-sided at $\alpha = 0.05$ (Appendix C.5). We score

$$\text{TRUTH} = \mathbf{1}[D{=}\text{disc}(Z)] \text{ or } \mathbf{1}[D{=}\text{ncd} \wedge Z{=}\varnothing],$$
$$\text{EVID} = \mathbf{1}[D{=}\text{disc}(H) \wedge p_H{<}\alpha] \text{ or } \mathbf{1}[D{=}\text{ncd} \wedge p_H{\geq}\alpha],$$

leaving both undefined for abstentions. CONS records whether D equals the decision E implies, and PROTO whether exactly one confirmation occurred and D cites it. *Joint valid success* is TRUTH $\wedge$ EVID $\wedge$ PROTO. A grader's *credit* is any function of these fields. TRUTH scores the benchmark's latent target: "no confirmed discovery" acts as an operational prediction of the null class, not a claim that no effect exists. On an effect task it is therefore scored incorrect even when it was the right reading of an underpowered test, which EVID records separately.

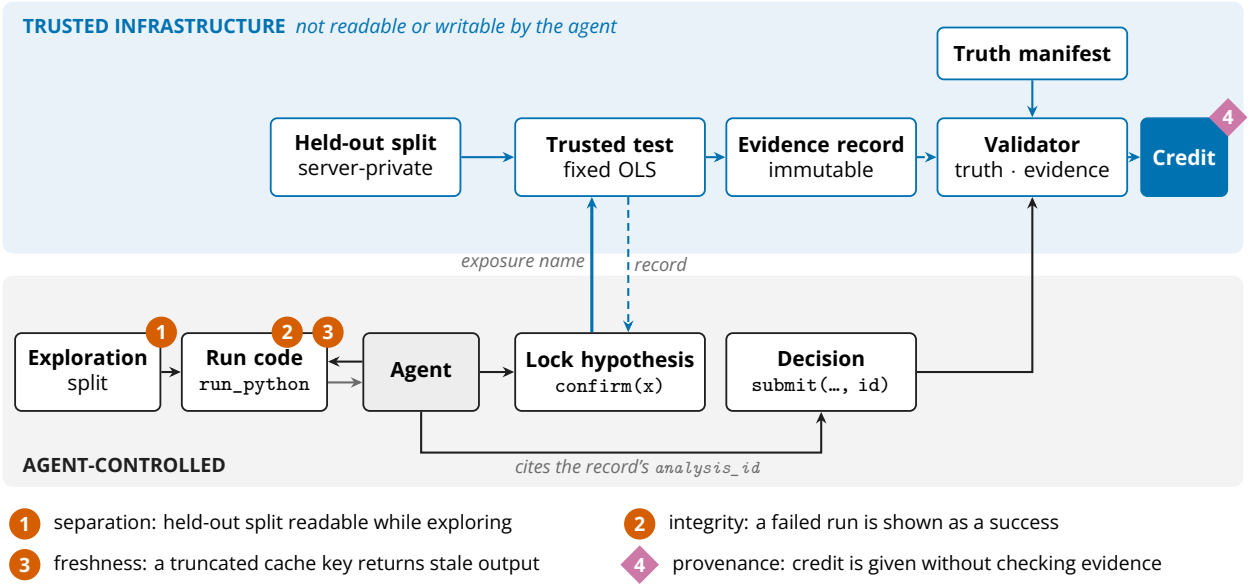


Figure 1 | **Trusted-confirmation pipeline (v2) and fault locations.** The agent explores freely but can only *name* one exposure for confirmation. A fixed test runs on server-private data and writes an immutable record. The decision cites that record, and the validator checks it against the truth manifest before credit is assigned. Appendix Figure C1 contrasts the v1 interface.

3.2 Two protocols

Original protocol (v1). `confirm(code)` ran an *agent-written* script on the confirmation split. The script printed a JSON `answer`, estimate, and p, and `submit` restated them. The grader credited truth-correct submissions that matched a re-execution of the cited script: equal `answer` labels, and for discoveries $p < 0.05$ with numbers within tolerance (Appendix B.1). This *agent evidence* certifies that a script reproducibly printed a claim, not that the claim follows from the intended test, so we call the v1 reference score *replay-graded success*. A script that only prints `{"answer":"x","p_value":0.001}` passes (a unit test).

Trusted protocol (v2). `confirm(exposure)` accepts one candidate name and nothing else. The first accepted call locks H. The harness then runs the oracle on a split that is never in the agent's workspace, and returns an `analysis_id` bound by SHA-256 to the dataset hash and the result. `submit(decision, analysis_id)` cites the record and restates no numbers. Fifteen unit tests check that code, extra fields, unknown exposures, repeat calls, and foreign or altered records are rejected, and that truth, evidence, and abstention are scored independently (Appendix C). Figure 1 shows the architecture.

3.3 Faults and comparisons

Each fault mutates one boundary (Appendix D). The v1 faults were coarser. LEAK exposed the split *and* allowed repeated confirmation; SILENT rewrote every run's status to "ok"; STALE keyed a cache on a script's first three lines. The v2 operators each change one thing:

- SEP_READ makes the split readable but keeps confirmation one-shot;
- INT_MASK shows only *failed* runs as `exit_code 0`;
- FRESH_CACHE keys the cache on a script's first 256 characters.

PROV (v1) and PROV_GRADE (v2) credit TRUTH alone. We compare three things (Appendix E): *same-trace regrading*, *fixed-action replay* of recorded actions in a faulted environment (a mechanical diagnostic), and *fresh runs*. Their differences are not a causal mediation decomposition. Intervals come from a null/effect-stratified task-cluster bootstrap ($B = 20{,}000$). The provenance regrade is the pre-declared primary estimand, and the fault contrasts are Holm-

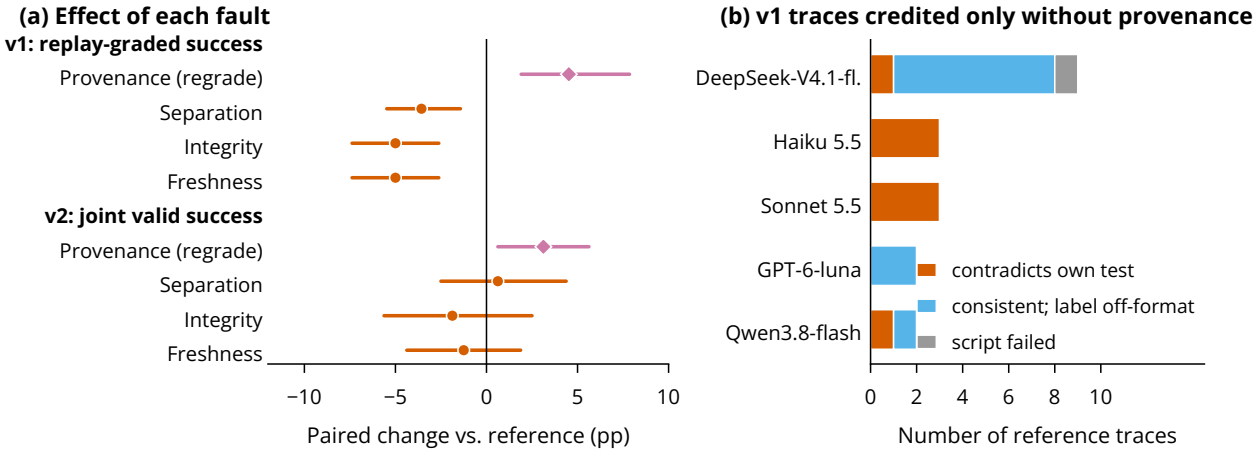


Figure 2 | **(a)** Paired change vs. the reference pipeline with 95% task-cluster bootstrap intervals. Diamonds regrade identical traces (change in credit); circles are fresh runs. v1 and v2 use different success constructs, so magnitudes are not comparable across blocks. **(b)** The 19 v1 reference traces credited only by the answer-only grader, by what the agent's own confirmation output showed. Full table: Appendix G.

adjusted secondary estimands (Appendix F).

4 Experimental Setup

v1. Seven OpenRouter models (October 2026; router defaults; listed in Table G3) ran on 20 tasks (10 effect, 10 null) with 3 repeats in REF, LEAK, SILENT, and STALE, for 1,680 runs and $35.60. PROV regrades the 420 REF traces. We re-analyze these records without new calls and re-execute all 1,665 cited confirmation scripts under instrumentation; every re-execution reproduced its recorded output. **v2.** Forty new tasks (seeds 5000–5039, 20 effect and 20 null) come from the unchanged generator and are frozen with SHA-256 manifests. Four v1 models (GPT-6.1-sol, Haiku 5.5, GPT-6-luna, DeepSeek-V4.1-flash; same snapshots) ran once per task in REF, SEP_READ, INT_MASK, and FRESH_CACHE, for 640 runs and $2.94. Endpoints were pinned, reasoning effort was `low`, and condition order was randomized within blocks (Appendix H). A deterministic *reference agent* screens all ten exposures with the oracle model on exploration data, locks the minimum-p exposure, confirms once, and submits the implied decision.

5 Results

5.1 Truth and evidence diverge in both directions

Applying the oracle to every recoverable hypothesis in the 420 v1 reference traces gives 271 conclusions that are true and supported, 30 that are *supported but false* (29 false discoveries with held-out $p < 0.05$ and one missed effect), and only 1 that is true but unsupported. Another 118 have no evidential status because no single hypothesis was locked (Appendix Figure B1). The v1 reference grade is therefore *replay-graded*, not the v2 construct: of its 364 credits, 262 are true and oracle-supported, 63 undetermined, 38 null-task abstentions, and 1 unsupported. The supported-but-false cell is concentrated in particular datasets. Of 116 v1 false discoveries, 114 fall on two null tasks whose confirmation split contains an oracle-significant exposure, and the deterministic reference agent also errs on both (Appendix A.6). Excluding them lowers the null false-discovery rate from 13.8% to 0.3%. On fresh simulated data the reference agent's rate is 4.8% (95% CI 4.3–5.5). The 13.8% therefore describes which 10 null datasets were drawn, each re-sampled 84 times; it is not agent calibration, although agents' selection policies still decide whether such associations get found and reported (§5.3). In v2, all 3 false discoveries in 80 null reference runs fall on the one task where the reference agent also errs. *Lesson: report false discoveries against a trusted baseline on the same datasets,*

and count datasets, not runs.

5.2 What an answer-only grader credits

Dropping the evidence check from the v1 grader raises credit on identical traces by $+4.5$ pp ($[+1.9, +7.9]$). The sign is fixed by construction, so the content is the composition (Figure 2b). **Eight traces contradict their own evidence.** The confirmation printed a significant exposure, and the agent submitted "none" or "inconclusive" on a null task. In Sonnet's case it cited a sign reversal from exploration, a defensible judgment outside the stated rule that was right only because the truth was null. **Ten are correct readings of non-significant tests that broke the output contract.** The instructions defined `answer` as the conclusion, but these scripts used it to name the tested candidate. A typical DeepSeek script printed `{"answer":"nitrate", "p_value":0.2015}`, and the agent submitted "none". The label check rejected a decision that its own evidence supports. **One** has a failed script. So 2.6 of the 4.5 pp reflects format compliance, not evidence violations (Appendix B.6); 8 of these 11 are DeepSeek's, and without DeepSeek the estimate is $+2.8$ pp. v2 removes the label problem, because decisions cite records, but PROV_GRADE still adds $+3.1$ pp ($[+0.6, +5.6]$). All 5 newly credited runs are Haiku declaring "no confirmed discovery" on null tasks without calling `confirm`, because exploration showed nothing worth testing (Appendix B.7). This lifts Haiku from last to first of four models, though on 5 runs of one model and in only 64% of bootstrap replicates. *Lesson: when half the tasks are null, "no discovery" is free unless credit requires evidence.*

5.3 One confirmation call is not one test

The v1 harness enforced one `confirm` *call*, but each call executed arbitrary code on the confirmation split. Several distinct quantities collapse under "confirm once": the number of calls, the number of candidates entering fitted models, the number of separate inferential tests, whether the reported candidate was chosen *using* confirmation results, and whether a target was fixed beforehand. Logging every `statsmodels`/SciPy fit inside the re-executed scripts recovers the second and third directly (Algorithm 1). Among the 1,665 scripts (Appendix Table B1), 56.2% fit one candidate, 31.4% fit several candidates in one model structure (typically all ten as joint regressors), 10.9% fit separate models to different candidates, and 1.6% have no identifiable fit. These are different statistical acts. A joint regression may still serve a single pre-specified coefficient, but separate fits are a screen on held-out data. In 46 of the 47 discoveries from separate-fit scripts, the reported exposure was the smallest-p candidate, and 18 of the 47 were false. 126 of the 182 separate-fit scripts contain a correction pattern such as Bonferroni; this is a lexical indicator, not a verified procedure, and the reported p was typically uncorrected. Locking a hypothesis on exploration data and testing it once discovers on 4.8% of nulls. Taking the minimum p over ten confirmation tests discovers on 39.4%, close to $1 - 0.95^{10} = 40.1\%$ for independent tests.

Single-test scripts matched the oracle's significance call in 793 of 794 cases: the arithmetic was right, but the mapping from tool call to inferential act was not fixed. v2 fixes it, since `confirm` takes a name, runs one test, and locks the hypothesis. Under v2, every confirmed decision equalled the decision its record implied, against 8 contradictions among v1 reference traces. Agents agreed with the deterministic reference agent on 99.4% of decisions and 95% of locked hypotheses. Its 11 supported-but-false reference runs are sampling and selection error, which no interface removes. *Lesson: pre-registration must constrain the hypothesis and the analysis, not the number of tool invocations.*

5.4 Environment faults matter only when they manifest

Observed effects. In v1, replay-graded success fell by 3.6 pp under LEAK ($[-5.5, -1.4]$), 5.0 pp under SILENT ($[-7.4, -2.6]$), and 5.0 pp under STALE ($[-7.4, -2.6]$). Holm-adjusted task-level sign-flip randomization tests give $p = 0.023$, 0.006, and 0.006. All three keep their sign under every leave-one-task-out refit and after removing the two dominant null tasks. Measured and

replay-graded success moved together (within 0.3 pp); replay agreement does not, however, establish independent evidential validity. The losses are concentrated: Qwen lost up to 21.7 pp and DeepSeek up to 16.7 pp, while GPT-6.1-sol, Sonnet, Gemini, and Haiku moved by at most 3.3 pp (Appendix F.5).

Replay. Re-executing the 420 reference action sequences unchanged in each faulted environment moved the grade by 0, 0, and -0.5 pp. This holds even though 26.7% of the sequences contained an error that SILENT would hide and 42.1% hit a stale entry under STALE. A fault changes the grade of fixed behaviour only if it touches the cited confirmation, which happened in 2 of 420 replays. The fresh-run losses are not mechanical. They are consistent with agents responding to altered observations, but the comparisons do not identify the mechanism; one Qwen trace shows such a response (Appendix E.4).

Manifestation. A scheduled fault is not a delivered treatment: masking needs a failing run, and staleness needs two scripts sharing a key prefix. v2 agents used about six tool calls per run, so a masked failure reached the agent in 15% of INT_MASK runs and a stale output in 19% of FRESH_CACHE runs. The corresponding v1 rates were 28% and 44%, under broader fault semantics. The held-out file was visible in 88% of SEP_READ runs but read in 4%, never before the lock.

Scope of inference. The v2 contrasts are $+0.6$, -1.9, and -1.3 pp, each with an interval of about ± 4 pp that includes zero (Holm $p \geq 0.56$). Runs in which a v2 fault manifested reached 83% (masked) and 80% (stale) joint validity, against 89% and 91% elsewhere. Manifestation is post-treatment behaviour, however, so these contrasts are descriptive (Appendix D.5). v1 and v2 differ in fault semantics, model panel, inference settings, repeats, and tasks, so the smaller v2 effects cannot be attributed to the trusted protocol (Appendix D.6). *Lesson: an injected defect is a treatment only to the extent that it reaches the agent. Report trigger, visibility, and use with every fault effect.*

6 Discussion

Scientific truth does not validate scientific process. Among v1 reference traces, 30 conclusions were supported and false, and 8 were true but contradicted their own evidence. Agreement with the generating process alone cannot separate a well-run non-replication from a lucky guess. Benchmarks should report TRUTH and EVID separately and judge false discoveries against a trusted agent run on the same datasets.

Grader strictness is not grader correctness. 11 of the v1 grader's 21 rejections in REF were decisions consistent with a non-significant test whose `answer` field named the tested candidate. The check enforced format under the name of evidence, and the answer-only grader erred the other way. A grader must encode the decision rule, as v2 does with typed decisions and record lookup.

Infrastructure defines the measured capability. Under unrestricted confirmation, correct inference, a screen of held-out data, and a lucky answer earn the same score. A v2 score certifies hypothesis selection and decision discipline, not statistical programming (Appendix I).

Limitations. The results come from one linear-inference family, 20 (v1) and 40 (v2) datasets, four v2 models with one repeat, and one implementation per fault. v1 used router defaults, and its sandbox let 2 of 1,680 runs read the held-out file (Appendix I).

7 Conclusion

A label-matching evidence check penalized correct inference, an answer-only grader rewarded untested null decisions, and one call permitted many tests. Fault effects depended on exposure; false discoveries concentrated in particular datasets. A defensible scientific-agent pipeline executes the confirmatory analysis itself, binds each decision to an immutable evidence record, and reports truth, evidential support, and credit as distinct quantities.

Ethics Statement

Only synthetic data were used, and agent code ran sandboxed without network access. The faults studied are common engineering failures.

References

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025.

Junjie Chen. 2026. When agent evaluation becomes unreliable: An empirical study of oracle failures and configuration-dependent measurement in an LLM agent benchmark. *arXiv preprint arXiv:2610.16067*.

Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, et al. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations*, 2025.

Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. 2015. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638. https://doi.org/10.1126/science.aaa9375.

Andrew Gelman and Eric Loken. 2014. The statistical crisis in science. *American Scientist*, 102(6): 460–465.

Xincheng He, Wanli Dong, Zhaoqiang Guo, Yan Liu, and Lei Xu. 2026. SSCBench: Evaluating the evidential validity of fault-injection tests for tool-using LLM agents. *arXiv preprint arXiv:2610.11514*.

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. AI agents that matter. *arXiv preprint arXiv:2407.01502*.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.

Bodhisattwa Prasad Majumder, Harshit Surana, Dhruv Agarwal, Bhavana Dalvi Mishra, et al. DiscoveryBench: Towards data-driven discovery with large language models. In *International Conference on Learning Representations*, 2025. https://arxiv.org/abs/2407.01725.

Samuel Messick. 1995. Validity of psychological assessment: Validation of inferences from persons' responses and performances as scientific inquiry into score meaning. *American Psychologist*, 50(9): 741–749. https://doi.org/10.1037/0003-066X.50.9.741.

Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. 2018. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606. https://doi.org/10.1073/pnas.1708274114.

Jiaqi Shao, Hanck Chen, Wei Zhang, Maxm Pan, and Bing Luo. 2026. Do agent benchmarks measure capability? protocol validity in the age of agentic AI. *arXiv preprint arXiv:2607.22368*.

Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. 2024. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *arXiv preprint arXiv:2409.11363*.

Joseph P. Simmons, Leif D. Nelson, and Uri Simonsohn. 2011. False-positive psychology: Undisclosed flexibility in data collection and analysis allows presenting anything as significant. *Psychological Science*, 22(11):1359–1366. https://doi.org/10.1177/0956797611417632.

Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, et al. PaperBench: Evaluating AI's ability to replicate AI research. In *International Conference on Machine Learning*, 2025.

Zhen Wang, Fan Bai, Zhongyan Luo, Jinyan Su, Kaiser Sun, Xinle Yu, et al. FIRE-Bench: Evaluating AI agents on the rediscovery of scientific insights. In *International Conference on Machine Learning*, 2026. arXiv:2602.02905.

Yuxuan Zhu et al. 2025. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*.

Appendix overview

The appendix documents the implementation and analysis behind every main-text claim.

- **A** the generator and its calibration (§3, §5.1);
- **B** the instrumented audit of v1 confirmation scripts (§5.2, §5.3);
- **C** the trusted protocol (§3.2);
- **D** fault operators and manifestation (§5.4);
- **E** replay;
- **F** statistical procedures;
- **G** full result tables;
- **H** reproducibility;
- **I** construct and external validity.

Unless stated otherwise, numbers come from the frozen files under `research_v2/results/`, and code paths refer to the repository. The v1 study is the original experiment (`scieval/`); v2 is the trusted-protocol experiment (`research_v2/`).

A Data-Generating Process and Statistical Calibration

A.1 Formal generative model

A task is indexed by an integer seed s and a kind $k \in \{\text{effect}, \text{null}\}$ (`scieval/tasks.py`, function `make_task`). A single generator $\mathcal{R}_s = \texttt{default_rng}(s)$ drives every random choice in the following order:

1. Choose the framing $f = s \bmod 4$, with ten exposure names $c_1, \ldots, c_{10}$, an outcome name, a site name, and four site labels.
2. Draw a permutation π of $\{1, \ldots, 10\}$. The decoy is $d = \pi_1$, the text-encoded exposure is $m = \pi_2$, and in effect tasks the true exposure is $j^* = \pi_3$.
3. In effect tasks draw $\beta \sim U(0.15, 0.25)$; in null tasks $\beta = 0$ and $Z = \varnothing$.
4. Draw the exploration split, then the confirmation split, each of $n = 400$ rows, by the same row model.

For row r of a split:

$$S_r \sim \text{Unif}\{1,2,3,4\}, \qquad \delta_{S_r} \in (-0.8, -0.3, 0.3, 0.8), \tag{1}$$

$$X_{rj} \overset{iid}{\sim} N(0,1) \quad (j \neq d), \qquad X_{rd} = U_{rd} + 1.2\,\delta_{S_r},\ U_{rd} \sim N(0,1), \tag{2}$$

$$Y_r^\star = \delta_{S_r} + \beta X_{rj^*} + \varepsilon_r, \qquad \varepsilon_r \sim N(0,1). \tag{3}$$

Recorded columns are affine rescalings rounded to three decimals: exposure $= \text{round}(10 + 2X_{rj}, 3)$ and outcome $= \text{round}(50 + 5Y_r^\star, 3)$. Site is stored as its text label. Then:

- $\lfloor n/100 \rfloor = 4$ randomly chosen outcomes are set missing;
- $\lfloor 3n/100 \rfloor = 12$ randomly chosen entries of exposure m are replaced by the string `"NA"`, so the column is stored as text.

Because the two splits are consecutive draws from $\mathcal{R}_s$, they are independent samples from the same distribution conditional on the task parameters (f, π, β). Every quantity that matters for inference is invariant to the affine rescaling, since t-statistics are scale-free.

This is a controlled statistical generating process, not a general observational causal problem. The only confounding path is the designed one ($S \to X_d$, $S \to Y$). Exposures other than the decoy are independent of site and of each other. All effects are positive. The correct analysis, OLS with site indicators, is known to the designers.

A.2 Why site adjustment matters

Let $\sigma_\delta^2 = \mathrm{Var}(\delta_S) = \frac{1}{4}(0.64 + 0.09 + 0.09 + 0.64) = 0.365$. In a null task the decoy has no direct effect, but its marginal regression slope on the latent scale is

$$\frac{\mathrm{Cov}(Y^\star, X_d)}{\mathrm{Var}(X_d)} = \frac{1.2\,\sigma_\delta^2}{1 + 1.44\,\sigma_\delta^2} = \frac{0.438}{1.526} = 0.287, \qquad \rho(Y^\star, X_d) = \frac{0.438}{\sqrt{1.365 \times 1.526}} = 0.304. \quad (4)$$

With $n \approx 396$ complete rows the unadjusted t-statistic is about $\rho\sqrt{n-2}/\sqrt{1-\rho^2} \approx 6.3$, so an unadjusted test rejects essentially always. The simulation agrees: 5,000 of 5,000 null pairs reject (Table A1).

The true exposure in an effect task has marginal correlation $\beta/\sqrt{1.365 + \beta^2} \approx 0.13$–$0.21$ and $t \approx 2.6$–4.2. An unadjusted screen therefore prefers the decoy. Simulation gives 100% decoy selection on nulls and only 3.6% correct discoveries on effect tasks when the naive screen is followed by an adjusted test. Conditioning on site removes the path, because $X_d \perp Y^\star \mid S$ when $\beta = 0$ or $d \neq j^*$. The site-adjusted test of the decoy then rejects at the nominal rate (4.7%). The locked test of the true exposure has site-adjusted standard error about $1/\sqrt{n}$ on the latent scale, which gives $t \approx \beta\sqrt{n} \approx 3.0$–$5.0$.

A.3 Exploration versus confirmation

Let $\mathcal{D}^E$ and $\mathcal{D}^C$ be the two splits. A policy picks $\hat{H} = g(\mathcal{D}^E)$, any function of exploration data however many comparisons it makes, and then tests $\hat{H}$ once on $\mathcal{D}^C$. Under the null, $\mathcal{D}^C \perp \mathcal{D}^E$, so $\Pr(p_{\hat{H}}(\mathcal{D}^C) < \alpha) = \alpha$. The single confirmatory test keeps its nominal size, which the simulation reproduces (4.8%). The guarantee fails whenever $\hat{H}$ depends on $\mathcal{D}^C$:

- the hypothesis is selected or changed after inspecting confirmation results;
- a confirmation script fits several candidates and reports the best;
- the smallest of several confirmation p-values is reported without adjustment.

In the last case, $\Pr(\min_j p_j < \alpha) = 1-(1-\alpha)^m$ if the m tests are independent, which gives 40.1% for $m = 10$. Our exposures are independent except for the decoy, and the tests share one outcome vector, so the simulation is the authoritative figure: 39.4% (Table A1). A multivariable regression that includes several candidates is a different object. If its inferential target is one pre-specified coefficient, covariates are an adjustment set, not additional tests. It becomes a multiple test only when the reported candidate is chosen from its coefficients after fitting.

A.4 Monte Carlo calibration

`research_v2/dgp_validation.py` draws 5,000 null pairs (seeds $2{,}000{,}000 + i$) and 5,000 effect pairs (seeds $3{,}000{,}000 + i$). These seeds are disjoint from all benchmark seeds, and every pair is generated by the unchanged `_simulate`. Each pair is evaluated by deterministic policies. Each policy locks a hypothesis on exploration data and decides on confirmation data by the protocol rule: discover if $p < 0.05$, otherwise ncd. The policies are:

- *fixed*: test a pre-specified non-decoy exposure π_4;
- *decoy*: test the decoy with and without site adjustment;
- *reference*: run ten site-adjusted exploration fits and lock the minimum p;
- *naive*: run ten unadjusted exploration fits and lock the minimum p, then confirm adjusted (*naive*) or unadjusted (*naive-raw*);
- *all ten*: run ten adjusted confirmation fits and report the minimum p.

A fast numpy path computes the oracle statistic and is asserted equal to the frozen oracle (§C.5) to 10^{-10} on 200 exposure-dataset combinations before the run. Intervals are Wilson 95% intervals over independent simulated pairs.

The fixed-hypothesis rejection rate of 5.0% (Table A1) shows that the oracle test is calibrated on this generator, including the missingness and text encoding. The reference agent's 4.8% shows that exploration-only selection followed by one locked test keeps nominal size. Both rates concern independent datasets, which repeated agent runs on a fixed benchmark are not (§A.6).

Table A1 | **Generator and protocol validation** (Monte Carlo; unit: independent simulated dataset pair; 5,000 null and 5,000 effect pairs; rates in %, Wilson 95% intervals).

Policy (lock on exploration, test on confirmation)	Null: any discovery	Effect: correct discovery
Fixed pre-specified non-decoy exposure	5.0 [4.4, 5.6]	—
Decoy, site-adjusted test	4.7 [4.2, 5.3]	—
Decoy, unadjusted test	100.0 [99.9, 100.0]	—
Reference agent (adjusted screen, one locked test)	4.8 [4.3, 5.5]	91.1 [90.3, 91.9]
Naive screen (unadjusted), adjusted confirmation	4.7 [4.2, 5.3]	3.6 [3.1, 4.2]
Naive screen, unadjusted confirmation	100.0 [99.9, 100.0]	3.4 [3.0, 4.0]
All ten candidates tested on confirmation, minimum p	39.4 [38.1, 40.8]	93.5 [92.8, 94.1]

A.5 Power and difficulty

Detection by the reference agent factorizes approximately as $\Pr(\text{lock } j^*) \times \Pr(p_{j^*} < \alpha \mid \text{lock } j^*)$. Both terms increase with $\beta\sqrt{n}$. Table A2 refines the effect range into eight bins. Correct discovery rises from 73.2% at $\beta \in [0.150, 0.163)$ to 99.2% at $[0.238, 0.250]$. At the weakest bin the screen selects the true exposure 85.2% of the time, and the remaining loss is confirmation power. The naive screen recovers the truth in at most 10.7% of pairs even at the largest effect, because the decoy's marginal signal ($t \approx 6.3$) exceeds the true signal.

Table A2 | **Detection by effect size** (Monte Carlo; 5,000 independent effect-task pairs; unit: simulated dataset pair; $n = 400$ per split). *Reference*: correct discovery by the reference agent; *Selected truth*: its exploration screen locks the true exposure; *Naive*: unadjusted screen, adjusted confirmation; *All ten*: minimum-p over ten confirmation tests.

β bin	Pairs	Reference (%)	Selected truth (%)	Naive (%)	All ten (%)
$[0.1500, 0.1625)$	635	73.2	85.2	0.5	83.1
$[0.1625, 0.1750)$	636	84.0	89.9	0.5	87.9
$[0.1750, 0.1875)$	662	89.9	93.8	1.2	92.9
$[0.1875, 0.2000)$	629	91.9	96.8	2.5	91.7
$[0.2000, 0.2125)$	644	96.0	97.2	2.8	96.9
$[0.2125, 0.2250)$	584	98.3	99.0	4.6	98.6
$[0.2250, 0.2375)$	603	98.0	99.0	6.6	98.7
$[0.2375, 0.2500)$	607	99.2	99.5	10.7	98.8

A.6 Concentration of v1 false discoveries

All runs on a task share one realization of $\mathcal{D}^C$. For two of the ten v1 null tasks that realization contains incidental site-adjusted associations (Table A3):

- `ecology-null-1011`: `nitrate` ($p = 0.0015$), `elevation_m`, and `rainfall_mm`.
- `clinical-null-1009`: `crp_level` ($p = 0.039$) and `hdl`.

On both, the minimum-p exploration candidate is the one that later appears significant (exploration $p = 0.066$ and 0.052). Any agent following the protocol well therefore locks a hypothesis that the confirmation sample happens to support, and the reference agent does exactly that. Table A4 shows which models named which exposure. Five of seven models discovered `nitrate` in at least 11 of 12 runs. On `clinical-null-1009`, Sonnet, Gemini, and Haiku

discovered nothing in 36 runs, while GPT-6.1-sol and GPT-6-luna discovered `crp_level` in all 24.

The 84 runs per task are conditional replicates on a fixed sample. They measure agent behaviour given that sample, not the sampling distribution of the test. The 13.8% run-level false-discovery rate is therefore a weighted average over ten fixed datasets, two of which carry a significant incidental association. Excluding them leaves 2 discoveries in 672 runs (0.3%), both from single runs on other tasks. These are ordinary chance associations at the stated α, not evaluator defects.

Table A3 | **False discoveries per v1 null task** (unit: run; 84 runs per task = 7 models $\times$ 4 configurations $\times$ 3 repeats). *Reference agent*: decision of the deterministic baseline on the frozen splits. *Oracle-significant*: candidates with site-adjusted $p < 0.05$ on the confirmation split.

Task	FDs	Exposures named	Reference agent	Oracle-significant in confirmation
`ecology-null-1011`	76	elevation_m, nitrate	nitrate	elevation_m, nitrate, rainfall_mm
`clinical-null-1009`	38	crp_level, vitamin_d	crp_level	crp_level, hdl
`clinical-null-1001`	1	crp_level	none	—
`clinical-null-1005`	1	bmi	none	—
other six null tasks	0	—	none	—

Table A4 | **Discoveries on the two dominant v1 null tasks, by model** (unit: run; 12 runs per model and task = 4 configurations $\times$ 3 repeats). Every run on a task shares one confirmation split.

Task	Model	Discoveries	Exposures named (count)
`ecology-null-1011`	GPT-6.1-sol	7/12	nitrate (7)
`ecology-null-1011`	Sonnet 5.5	12/12	nitrate (12)
`ecology-null-1011`	Gemini 4 Argon	12/12	nitrate (12)
`ecology-null-1011`	Haiku 5.5	12/12	nitrate (7), elevation_m (5)
`ecology-null-1011`	GPT-6-luna	12/12	nitrate (12)
`ecology-null-1011`	DeepSeek-V4.1-fl.	10/12	nitrate (9), elevation_m (1)
`ecology-null-1011`	Qwen3.8-flash	11/12	nitrate (10), elevation_m (1)
`clinical-null-1009`	GPT-6.1-sol	12/12	crp_level (12)
`clinical-null-1009`	Sonnet 5.5	0/12	—
`clinical-null-1009`	Gemini 4 Argon	0/12	—
`clinical-null-1009`	Haiku 5.5	0/12	—
`clinical-null-1009`	GPT-6-luna	12/12	crp_level (12)
`clinical-null-1009`	DeepSeek-V4.1-fl.	11/12	crp_level (10), vitamin_d (1)
`clinical-null-1009`	Qwen3.8-flash	3/12	crp_level (3)

B Instrumented Confirmation and Evidence-Oracle Audit

B.1 The v1 confirmation interface and its scoring contract

The v1 system prompt (`scieval/agent.py`) defined three tools.

- `run_python(code)` runs a script in a fresh process whose working directory contains `data/explore.csv`.
- `confirm(code)` runs a script in a separate directory containing `data/explore.csv` and `data/confirm.csv`. The instructions state that *"the script must print exactly one line of JSON of the form {"answer": <exposure name, "none", or "inconclusive">, "estimate": <float or null>, "p_value": <float or null>}"*, and that confirm may be called once.
- `submit(answer, estimate, p_value, evidence_run_id, rationale)` ends the run.

The contract therefore defines `answer` as the *conclusion*.

The grader (`scieval/grade.py`) proceeds in four steps.

1. It normalizes answers: case- and punctuation-insensitive exposure matching, `none`, `inconclusive`, otherwise `other`.
2. It selects the cited confirmation: the event whose run identifier equals `evidence_run_id`, or else the last confirmation.
3. It declares the submission *supported by output* o iff (i) o has exit code 0; (ii) the last stdout line that parses as JSON with an `answer` key exists; (iii) its normalized answer equals the submitted one; and (iv) if that answer is an exposure, the printed $p < 0.05$, $|p_{\text{sub}} - p_{\text{out}}| \leq \max(10^{-3}, 0.05\, p_{\text{out}})$, and $|\hat{\beta}_{\text{sub}} - \hat{\beta}_{\text{out}}| \leq \max(10^{-3}, 0.02\, |\hat{\beta}_{\text{out}}|)$.
4. It assigns three scores:
   - *measured* uses the pipeline's recorded output;
   - *valid* uses a reference re-execution of the cited script;
   - *correct* means the answer equals the truth, or the task is null and the answer is `inconclusive`.

PROV credits *correct* alone. Two consequences follow.

- The tolerances check transcription of the agent's own numbers, not the statistical computation that produced them.
- On null tasks v1 counted "inconclusive" as correct, so v1 abstentions were creditable. 38 of 46 REF abstentions were credited (Figure B1). The v2 validator scores abstention separately.

B.2 Instrumentation

`research_v2/audit_existing.py` re-executes, for each of the 1,680 runs, the cited confirmation script. It uses the original macOS sandbox profile and a fresh copy of both splits, and wraps the script with `research_v2/audit_runner.py`. Before the script runs, the wrapper replaces several entry points with logging versions:

- `statsmodels.regression.linear_model.RegressionModel.fit` and `statsmodels.genmod.generalized_linear_model.GLM.fit`, which cover OLS, WLS, GLS, the formula API, and GLMs;
- `scipy.stats.linregress`, `pearsonr`, and `spearmanr`.

It then runs the unmodified script with `runpy`. Each fitted model's design-matrix columns are mapped to candidate exposures by Algorithm 1, and the log is written at interpreter exit. The re-executed JSON equals the recorded JSON in all 1,665 runs with a confirmation, so the instrumented executions are the recorded ones. No abstract-syntax-tree analysis is used. Arithmetic done by hand in numpy is invisible to the hooks, which leaves the 26 "no identifiable fit" scripts.

B.3 Identifiability and denominators

The tested hypothesis H is defined as follows.

- For a discovery, H is the claimed exposure. The claim itself names the hypothesis, whatever else the script fitted.
- For "none", H is the unique candidate that entered any fit, when exactly one did.
- Otherwise H is unrecoverable.

We never impute H from the truth. Of 1,680 v1 runs, 1,665 cited a confirmation and 1,185 have a recoverable H. The 480 unrecoverable runs are 252 "none" decisions from multi-candidate scripts, 187 "inconclusive", and 41 with no usable submission. In the 420 REF traces, 417 cite

Algorithm 1 Recovering tested candidates from one re-executed v1 confirmation script

Require: script P; confirmation table C; candidates $c_1..c_{10}$; $F_j = \{\text{round}(v, 3) : v \in C[c_j]\}$

install logging hooks on model fitting; run P unchanged
for each logged fit with design columns (name_k, v_k), row labels R, p-values p_k **do**
  for each column k **do**
    if c_j is a substring of name_k (case-insensitive) **then**
      map $k \mapsto c_j$ ▷ name
    else if v_k is constant or binary **then**
      skip ▷ intercept, site dummies
    else if $> 90\%$ of the first 200 rounded values of v_k lie in F_j **then**
      map $k \mapsto c_j$ ▷ fingerprint
    else if $|R| = |v_k|$ and $|\text{corr}(v_k, C[c_j][R])| > 0.999$ **then**
      map $k \mapsto c_j$ ▷ standardized copy
    else
      mark k unidentified
return per-fit sets of mapped candidates and their p-values

a confirmation and 302 have a recoverable H. All evidence comparisons are restricted to recoverable runs and state their denominators.

B.4 A taxonomy of confirmatory structure

Each script falls into one of four classes (Tables B1–B2):

- *single candidate*: one candidate enters every fit;
- *several candidates, one model structure*: several candidates enter fits, but the per-fit candidate sets are identical, typically one regression with all ten exposures;
- *separate per-candidate fits*: at least two fits involve different candidate sets, which is a screen;
- *no identifiable fit*.

The 42% figure in the main text is the union of the second and third classes, the scripts in which more than one candidate was fitted. The 10.9% figure is the third class alone. They measure different things. The second class is compatible with a pre-specified target coefficient plus covariate adjustment. GPT-6.1-sol, which uses it in 71% of scripts, almost always fits all ten exposures jointly.

Selection on confirmation data cannot be read from code structure alone. We therefore report an observable indicator: the reported discovery is the smallest-p candidate among those fitted. It holds for 46 of 47 discoveries from separate-fit scripts and for 292 of 329 from joint models. In joint models this is expected whenever the target coefficient is also the strongest, so the indicator is informative mainly for separate fits. A lexical scan for multiplicity corrections (Bonferroni, Holm, `multipletests`, FDR, α/m) matches 126 of 182 separate-fit scripts. One Gemini script screened all ten exposures and then applied a Bonferroni threshold to the minimum p (Trace 3). Such a screen controls the family-wise error, and its decision rule is valid at level α. The protocol, however, asked for a single pre-specified test, and the script reported the unadjusted p.

B.5 Agent evidence versus oracle evidence

For each run with recoverable H we compare four quantities: *agent evidence* (the v1 support check against re-executed output), *oracle evidence* (the frozen test of H), the hidden truth, and v1 credit. Agent and oracle evidence agree in 96.3% of the 1,185 runs (98.5% for discoveries,

Table B1 | **v1 confirmation scripts by inferential structure** (all 1,665 cited confirmation scripts across REF, LEAK, SILENT, STALE; unit: run). *Null disc.*: discoveries on null tasks (all false). *Min-p*: discoveries whose reported exposure is the smallest-p candidate among those fitted (only defined with ≥ 2 candidates). *Correction*: script text contains a multiplicity-correction pattern (Bonferroni, Holm, `multipletests`, FDR, α/m); a lexical indicator, not a verified procedure.

Structure	Scripts	%	Discoveries	Null disc.	Min-p	Correction
Single candidate	935	56.2	556	50	0	3
Several candidates, one model structure	522	31.4	329	44	292	44
Separate per-candidate fits	182	10.9	47	18	46	126
No identifiable fit	26	1.6	11	4	0	3

Table B2 | **Share of v1 confirmation scripts (%) in each structure, by model** (unit: run; denominator: runs with a cited confirmation, all four v1 configurations).

Model	Scripts	Single	Multivariable	Separate fits	No fit
GPT-6.1-sol	240	29	71	0	0
Sonnet 5.5	240	88	10	2	0
Gemini 4 Argon	240	63	0	36	0
Haiku 5.5	240	44	53	2	0
GPT-6-luna	240	63	37	0	0
DeepSeek-V4.1-fl.	230	53	35	8	4
Qwen3.8-flash	235	52	13	29	6

87.6% for "none"). Of the 44 disagreements, 41 have agent evidence false and oracle evidence true, and 3 the reverse. By the agent's printed output, the 44 split as follows:

- 29 label mismatches (printed $p \geq 0.05$, but the `answer` field named a candidate or used another label);
- 6 printed a significant result;
- 5 failed scripts;
- 4 printed no p-value.

Two of the 3 agent-yes/oracle-no cases are null-task discoveries in which the agent's own analysis crossed α and the oracle's did not. DeepSeek fitted a joint model of all ten exposures ($p = 0.030$ against oracle 0.081), and GPT-6-luna used a differently specified model (0.037 against 0.053). For single-candidate scripts, the agent's significance call matched the oracle in 793 of 794 runs. Agents almost never computed wrong statistics. Disagreements arise from how outputs were labelled and from which model was fitted.

B.6 Why the reference grader rejected decisions

The 21 REF traces that the v1 reference grader rejected for lack of support, although a submission existed, fall into four relations between the decision and the agent's own printed output:

- *contradiction* (8): an exposure was printed with $p < 0.05$, and the agent submitted "none" or "inconclusive";
- *consistent with a contract violation* (11): $p \geq 0.05$ was printed with an `answer` that names a candidate or differs in label; the submitted "none" or "inconclusive" is the decision the evidence implies;
- *failed script* (2).

The protocol defines `answer` as the conclusion, so the 11 are genuine violations of the output contract, and a format checker is right to reject them. A provenance check exists to certify that the decision follows from the evidence, however, and on that criterion they pass. The v1

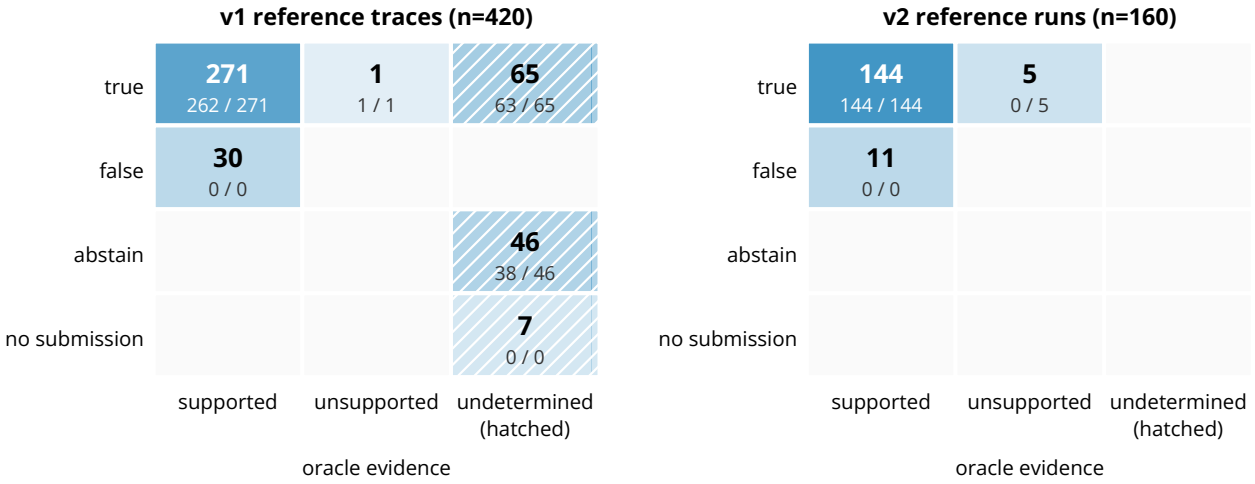


Figure B1 | **Truth × oracle evidence for reference traces, with credit under each grader.** Unit: run. Each cell gives its count and, beneath it, "a / b": credit under the reference grader / under the answer-only grader. Hatched cells are *undetermined*: no single locked hypothesis exists, which is different from evidence that fails to support the decision. In v1 the reference grader credits 262 of 271 true-and-supported traces. The 9 exceptions are evidentially consistent decisions that broke the output contract. It also credits 38 abstentions, because v1 counted "inconclusive" as correct on null tasks. In v2 the 5 true-but-unsupported runs are Haiku's untested "no confirmed discovery" decisions.

grader therefore merged two properties, format compliance and evidential support, and the answer-only grader merged two others, luck and support.

The graders in this paper implement four policies:

- v1 reference: *correct* ∧ format-and-number agreement;
- v1 PROV: *correct*;
- v2 reference: TRUTH ∧ EVID ∧ PROTO, where EVID reads a typed decision against a looked-up record;
- v2 PROV_GRADE: TRUTH.

Only the v2 reference policy separates the four properties.

B.7 Representative traces

All traces are real. Run identifiers name the frozen file `<model>/<config>/<task>_r<repeat>`. Code is abbreviated where marked [...]; printed values are exact.

Trace 1: a consistent decision rejected over the label
(`deepseek-v4.1-flash/REF/ecology-null-1003_r1`).

- *Exploration:* nitrate is the only exposure significant after region adjustment ($\hat{\beta} \approx 0.30$, $p = 0.022$).
- *Confirmation:* the script fits `species_richness ~ C(region) + nitrate` and prints `{"answer": "nitrate", "estimate": -0.16137062319236306, "p_value": 0.20153068810622493}`.
- *Submission:* `answer="none"`, same numbers; the rationale says the effect "did not replicate".
- *Grading:* v1 reference not credited (label mismatch). Answer-only credited. Oracle: $p(\text{nitrate}) = 0.2015$, so "no confirmed discovery" is supported. Truth: null.

Trace 2: a true decision that contradicts its evidence
(`claude-sonnet-5.5/REF/clinical-null-1009_r0`).

- *Confirmation output:* `{"answer": "crp_level", "estimate": 0.2500235772339945, "p_value": 0.03949806053629611}`.

- *Submission:* `answer="inconclusive"`, with the rationale *"In the held-out data the hospital-adjusted slope was $+0.25$ ($p = 0.039$), which is the opposite sign [to exploration]. The direction did not replicate, so the confirmatory p-value just under 0.05 doesn't support a stable direct effect."*
- *Grading:* the oracle agrees ($p = 0.0395$), so under the stated rule the evidence implies a discovery. Truth is null, so the answer-only grader credits the run, and the v1 reference grader does not.

The agent applied a sign-replication criterion the protocol did not state.

Trace 3: one call, ten tests
(`gemini-4-argon/REF/ecology-null-1011_r0`).

- *Confirmation code (abbreviated):* `for exp in exposures: mod = smf.ols(f'species_richness ~ {exp} + C(region)').fit(); p_vals[exp] = mod.pvalues[exp] … best_exp = min(p_vals, key=p_vals.get); bonf_p = min_p * 10;` [...].
- *Output:* `{"answer": "nitrate", "estimate": 0.4200009765401044, "p_value": 0.001488251386534853}`.
- *Grading:* the instrumentation logs ten separate fits plus a joint model. The report is the minimum-p candidate with the unadjusted p; the Bonferroni-adjusted value, 0.0149, also crosses α. Truth: null, so this is a false discovery.

Trace 4: supported but false
(`claude-sonnet-5.5/REF/ecology-null-1011_r0`).

- *Exploration:* nitrate is the strongest region-adjusted association ($p \approx 0.07$).
- *Confirmation:* a single locked test, `smf.ols('species_richness~nitrate+C(region)', d)`, prints `{"answer": "nitrate", "estimate": 0.4200009765401044, "p_value": 0.001488251386534853}`.

The protocol was followed exactly, the evidence supports the claim, and the claim is false. This is the cell the reference agent also lands in.

Trace 5 (v2): a null decision without a test
(`claude-haiku-5.5/REF/ecology-null-5031_r0`). The first exploration run prints summaries; the second fits each exposure with region adjustment and prints p-values of 0.34–0.98 for all ten. The agent then submits `decision="no_confirmed_discovery"`, `analysis_id=None`, with the reason: *"No exposure showed a direct-effect signal worth pre-registering, so I did not spend the single confirmatory test."* The validator records TRUTH true, EVID false, and PROTO false, so the reference grader gives no credit and the answer-only grader gives credit. The decision is a reasonable screening rule, but it is not the protocol, and on a null task it cannot be distinguished from not doing the work.

B.8 Two classes of evaluator error

The audit separates two kinds of failure.

- *Enforcement errors*: the interface does not enforce the intended procedure, so a script can screen held-out data or a submission can restate numbers it never obtained.
- *Interpretation errors*: the grader misreads evidence that was produced, for example through a label comparison or an answer-only rule.

Appendix C shows how v2 addresses each. Sampling error is outside both classes.

C Trusted Confirmation Protocol and Implementation

C.1 Trust boundaries

Object	Readable by agent	Writable by agent
Exploration split (`ws/explore/data/explore.csv`)	yes	yes (its own copy)
Confirmation split (task store)	no: absent from workspace; sandbox denies reads	no
Truth manifest (`task.json`)	no: same denial	no
Oracle code and evidence registry (harness memory)	returned copy of own record only	no
Submission	—	yes (once; ends the run)
Validator and grader state	no	no

Agent code runs under `sandbox-exec`. The profile allows no network, confines writes to the run workspace and temporary directories, and in v2 adds `(deny file-read* (subpath <task store>))`. The confirmation split is never copied into the workspace except under SEP_READ. The v1 profile lacked the read denial, and v1 kept the confirmation split in a sibling directory of the exploration workspace. Two v1 runs reached it by relative path (§D.2).

C.2 Tool interfaces

The tool schemas (`research_v2/agent2.py`) are:

- `run_python{code: string}`, which returns `event_id`, `exit_code`, `stdout`, and, when non-empty, `stderr`, each truncated at 4,000 characters;
- `confirm{exposure: string}`;
- `submit{decision: enum[discover, no_confirmed_discovery, abstain], exposure, analysis_id, reason}`.

A valid confirmation request is `{"exposure":"soil_moisture"}`. Its response, verbatim from `gpt-6.1-sol/REF/ecology-null-5007_r0`, is

```
{"analysis_id": "ecology-null-5007:c1:d1f11f5626b9", "exposure": "soil_moisture",
 "estimate": -0.13376540198128883, "std_error": 0.1139318312133778,
 "test_statistic": -1.174082787546578, "p_value": 0.24107647887808165, "n": 396,
 "alpha": 0.05, "adjustment": ["region"], "test_family": "ols_two_sided_classical_se"}
```

`Harness.confirm` rejects requests in a fixed order:

1. any key other than `exposure`, e.g. `{"exposure":"x","code":"…"}`;
2. a value that is not one of the ten candidate names, which covers code and paths such as `"data/confirm.csv"`;
3. any request after a hypothesis is locked.

A rejected request is logged as `confirm_rejected` and returns an error string. It neither locks a hypothesis nor consumes the confirmation.

C.3 Confirmation as a state machine

The harness has no explicit state enum. State is the pair $(\texttt{locked}, \texttt{submission})$, both initially empty, and the agent loop stops when `submission` is set or after 20 model calls. Three states follow:

- OPEN: $(\varnothing, \varnothing)$;
- LOCKED: $(H, \varnothing)$;
- SUBMITTED: $(\cdot, D)$, which is terminal.

The transitions are:

State	Event	Result
OPEN	`run_python`	OPEN (event logged)
OPEN	valid `confirm(x)`	LOCKED(x); record issued
OPEN/LOCKED	malformed `confirm`	unchanged; `confirm_rejected`
LOCKED	`confirm(y)`, any y	unchanged; `confirm_rejected`
LOCKED	`run_python`	LOCKED
OPEN/LOCKED	`submit(D)`	SUBMITTED; run ends
any	20 model calls without submission	run ends; status `no_submission`

`submit` accepts any payload. Legality is judged afterwards by the validator (§C.6), so an illegal submission is recorded and scored rather than blocked. Under the optional SEP_REPEAT operator (implemented, not run), LOCKED accepts further confirmations without changing the lock.

Algorithm 2 Trusted one-shot confirmation (`Harness.confirm` and `oracle.evidence_record`)

Require: request a; task $(\text{id}, \text{outcome}, \text{site}, \text{candidates})$; registry $\mathcal{R}$; lock L

if keys$(a) \neq \{$`exposure`$\}$ **or** a.`exposure` $\notin$ candidates **or** ($L \neq \varnothing$ **and not** SEP_REPEAT) **then**
  log `confirm_rejected`; **return** error
$r \leftarrow$ REFERENCETEST(`read_csv`(confirm.csv), outcome, site, a.`exposure`)
r.task_id $\leftarrow$ id; r.confirmation_dataset_sha256 $\leftarrow$ SHA256(bytes of confirm.csv)
$h \leftarrow$ SHA256(`json.dumps`(r, `sort_keys`)); r.record_sha256 $\leftarrow h$
r.analysis_id $\leftarrow$ id $\|$ `:c` $(|\mathcal{R}|+1)$ $\|$ `:` h[:12]
$\mathcal{R}[r$.analysis_id$] \leftarrow r$; **if** $L = \varnothing$ **then** $L \leftarrow a$.`exposure`
return the public fields of r (dataset hash and record hash are withheld)

C.4 Evidence records

A record contains the following fields:

- the test result: exposure, estimate, standard error, t, p, residual degrees of freedom, n, α, the significance flag, the adjustment set, the test family, and the oracle version `ols-site-v1`;
- the task identifier and the SHA-256 of the confirmation file's bytes;
- `analysis_id` and `record_sha256`, as in Algorithm 2.

`verify_record` recomputes the digest over all other fields and checks it against `record_sha256` and the `analysis_id` suffix. This detects any later edit to a stored record. It is an integrity check inside a trusted process, not a cryptographic guarantee against an adversary who can write the registry, and records are not signed. A submission is linked when its `analysis_id` is the identifier of this run's locked record. A record issued in another task cannot be linked, because the registry is per run and the validator checks `task_id`. This is tested with a foreign identifier.

C.5 The trusted statistical computation

`reference_test` (`research_v2/oracle.py`) computes the test as follows. Let $y =$ `to_numeric`(outcome) and $x =$ `to_numeric`(exposure), with non-numeric strings becoming missing, and keep rows where y, x, and site are all present. Site levels are sorted, the first is the baseline, and $D \in \{0,1\}^{n\times 3}$ holds treatment-coded indicators. With design $X = [\mathbf{1}, x, D] \in \mathbb{R}^{n\times 5}$:

$$\hat{b} = (X^\top X)^{-1} X^\top y, \quad \hat{\sigma}^2 = \frac{\|y - X\hat{b}\|^2}{n-5}, \quad \text{SE} = \sqrt{\hat{\sigma}^2\,[(X^\top X)^{-1}]_{22}}, \quad t = \hat{b}_2/\text{SE}, \quad p = 2\,\bar{F}_{t,n-5}(|t|). \tag{5}$$

The assumptions are a linear conditional mean, homoskedastic errors, and the t reference distribution. All hold by construction for this generator, except that the outcome is rounded to three decimals. An unknown exposure raises an error, which `confirm` pre-empts. The result equals `statsmodels` `ols(y ~ x + C(site))` to relative error 10^{-9} (unit test).

C.6 Independent scoring functions

`validate` (`research_v2/harness.py`) first sets a status:

- `no_submission`;
- `malformed_decision`, if the decision is not in the enum;
- `malformed_exposure`, for a discovery naming a non-candidate;
- otherwise `complete`.

With locked record R (if any), implied decision $I =$ disc if $p_R < \alpha$ else ncd, and link flag ℓ (the cited id equals R's id):

$$\begin{aligned}\text{PROTO} = \text{complete} \wedge (\#\text{accepted confirms} = 1) \wedge \text{verify}(R) \\ \wedge \big(D{=}\text{abstain} \ \vee \ (\ell \wedge (D{\neq}\text{disc} \vee x{=}H))\big)\end{aligned} \tag{6}$$

$$\text{TRUTH} = \begin{cases} \mathbf{1}[x = Z] & D = \text{disc}(x) \\ \mathbf{1}[Z = \varnothing] & D = \text{ncd} \\ \text{undefined} & D = \text{abstain or incomplete} \end{cases} \tag{7}$$

$$\text{EVID} = \begin{cases} \ell \wedge x{=}H \wedge I{=}\text{disc} & D = \text{disc}(x) \\ \ell \wedge I{=}\text{ncd} & D = \text{ncd} \\ \text{undefined} & \text{otherwise} \end{cases} \tag{8}$$

CONS is defined when a record exists, as $D{=}I \wedge (D{\neq}\text{disc} \vee x{=}H)$. Joint valid success, which equals reference credit, is TRUTH $\wedge$ EVID $\wedge$ PROTO, with undefined counted as false. PROV_GRADE credit is TRUTH, again with undefined counted as false.

These fields differ in what they need: TRUTH needs only the manifest, EVID needs the record, CONS needs no truth, and PROTO needs the event log. The decision table for complete, protocol-valid runs is:

Task	Decision	Locked evidence	TRUTH	EVID	CONS	Ref. credit
effect	disc(j^*)	$p < \alpha$	1	1	1	1
effect	disc(j^*)	$p \geq \alpha$	1	0	0	0
effect	ncd	$p \geq \alpha$	0	1	1	0
effect	disc($x \neq j^*$)	$p < \alpha$	0	1	1	0
null	disc(x)	$p < \alpha$	0	1	1	0
null	ncd	$p \geq \alpha$	1	1	1	1
null	ncd	$p < \alpha$	1	0	0	0
any	abstain	any	–	–	0	0

An ncd without any confirmation (Trace 5) has TRUTH$= 1$ on null tasks, EVID$= 0$, and PROTO$= 0$.

C.7 Unit tests

All tests are in `research_v2/tests/test_protocol.py`; all 15 pass under `pytest`.

Test	Invariant
`oracle_matches_hand_fixture`	OLS estimate equals a hand-built least-squares fit on an 8-row fixture; n and degrees of freedom correct
`oracle_matches_statsmodels`	p and $\hat{\beta}$ equal `statsmodels` for all ten candidates (10^{-9})
`oracle_drops_only_incomplete_rows…`	complete cases are formed per tested exposure; `"NA"` strings coerced
`evidence_record_hash_and_tamper_detection`	issued record verifies; altering p fails verification
`fabricated_json_passes_v1…_but_not_v2`	a print-only JSON passes the v1 support check; v2 evidence equals the oracle's and the decoy claim is truth-incorrect
`confirm_rejects_code_paths_unknown…`	extra fields, paths, unknown names, and second calls rejected; one record; lock unchanged
`heldout_hidden_in_ref…`	held-out file absent in REF, unreadable by absolute path; present under SEP_READ, where confirm stays one-shot
`analysis_id_cannot_be_exchanged…`	a foreign task's id fails protocol and evidence
`truth_and_evidence_are_scored_separately`	true $\wedge$ supported credited; ncd against $p < \alpha$ unsupported; naming the truth without locking it is credited only by PROV_GRADE
`null_decision_and_abstention_…`	abstention has undefined truth and evidence and no credit
`prov_grade_differs_only_in_grader`	on a valid trace both graders agree
`int_mask_changes_only_failed_runs`	successful output byte-identical; failure shown as exit 0 without stderr; true exit logged
`fresh_cache_serves_stale…`	stale only on a shared 256-character prefix; no cache in REF
`faults_leave_confirm_and_grader_unchanged`	identical record (id, $\hat{\beta}$, p) under all four conditions
`fault_manifestation_records_trigger`	masked failure before the lock is detected; no trigger without failure

C.8 What the interface rules out, and what it does not

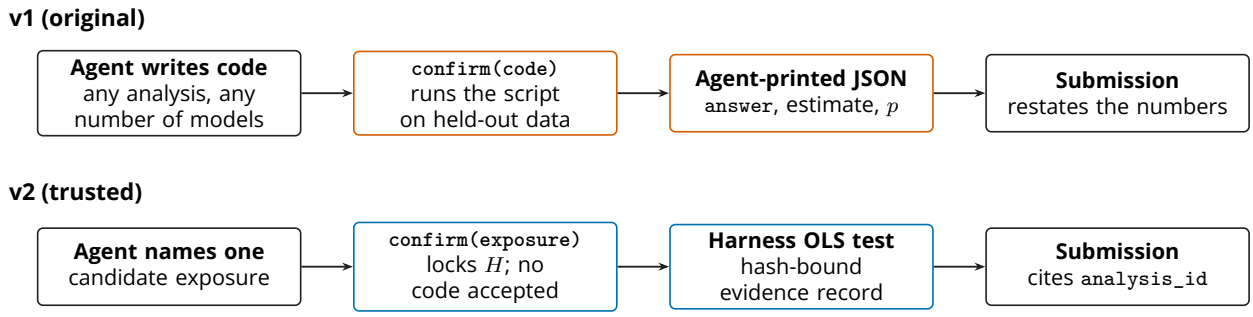


Figure C1 | **Confirmation interface in v1 and v2.** In v1 the agent's code ran on held-out data and produced the evidence that the grader then compared with the submission. In v2 the agent supplies only a candidate name; the harness runs one fixed test and the submission cites the resulting record.

By construction, v2 rules out four things:

- agent-authored confirmatory statistics;
- more than one confirmatory test per run;
- choosing the hypothesis after seeing a confirmation result, because the lock precedes the result;
- restated or fabricated numbers in the submission.

It does not prevent selecting a weak hypothesis, sampling error in the confirmation split, decisions made without confirmation (which are detected and scored, not blocked), or reading

held-out data when a fault exposes it. It also does not make agents scientifically competent. It makes the score attributable.

D Fault Operators and Manifestation

D.1 Fault model

We model an evaluation environment as a tuple $\mathcal{E} = (\mathsf{A}, \mathsf{X}, \mathsf{K}, \mathsf{C}, \mathsf{S}, \mathsf{G})$ with six components:

- A, the data-access policy;
- X, the execution-response adapter, mapping an execution result to what the agent sees;
- K, the cache and freshness semantics;
- C, the confirmation service;
- S, the evidence store;
- G, the grading policy.

A fault operator replaces one component by a specified mutation. Each operator has a reference invariant that the mutation violates. The machine-readable contracts are in `research_v2/faults/contracts/*.json`, with fields `intended_invariant`, `modified_boundary`, `trigger`, `visible_effect`, `unchanged`, and `tests`.

D.2 v1 faults

LEAK replaced A by copying the confirmation split into the exploration directory. It also replaced C by removing the one-confirmation refusal, so it changed two protections. SILENT replaced X for *every* run. The recorded exit code became 0, stderr was dropped, and the rendered status line read `status: ok` instead of `exit_code: k`, so even successful runs looked different from REF. STALE replaced K for both `run_python` and `confirm`. The key was the first three non-empty, non-comment, whitespace-stripped lines, and a hit returned the stored result of the earlier script. A hit is *stale* when the earlier code differs.

The v1 sandbox also permitted reads outside the workspace, and the confirmation split sat in a sibling directory. Two v1 runs, both Qwen and neither under LEAK, read it through `../confirm/data/confirm.csv`. One appears in §E.4. v2 removes the file from the workspace tree and denies reads of the task store.

D.3 v2 operators

SEP_READ changes A only. `confirm.csv` is copied to `explore/data/`, and confirmation remains server-side and one-shot. *Visible* means the filename appears in any `run_python` stdout or code. *Used* means code references `confirm.csv`, and we additionally record whether such a read precedes the hypothesis lock. A read after the lock cannot influence which hypothesis is tested and cannot produce a second test. It can only influence the decision, and the validator scores that against the record.

INT_MASK changes X only, and only when the true exit code is non-zero. The agent then sees `exit_code: 0`, any stdout produced before the failure, and no stderr. The true result stays in the event's `actual` field. Successful runs are byte-identical to REF, which is tested.

FRESH_CACHE changes K only. The key is $\mathrm{SHA256}(\mathrm{code}[:256])$. On a miss the result is stored; on a hit the stored result is returned. A hit is *stale* when the stored code differs from the submitted code, which requires two different scripts sharing their first 256 characters. Agents commonly reuse an import and data-loading header, which makes such collisions possible.

PROV_GRADE changes G only. Credit is TRUTH, with no conjunct for EVID or PROTO. It is applied to the identical REF traces, so behaviour is unchanged by definition. Because it removes conjuncts from a conjunction, it can only add credit: $\Delta = \Pr(\mathsf{TRUTH} \wedge \neg(\mathsf{EVID} \wedge \mathsf{PROTO})) \geq 0$.

D.4 Manifestation as an event sequence

For each faulted run, `fault_manifestation` records:

- *scheduled*: the condition is not REF;
- *triggered*: the mutation executed (always for SEP_READ; a failing run for INT_MASK; a stale hit for FRESH_CACHE);
- *visible*: a mutated observation was delivered;
- *used*, under an operational definition: held-out file referenced in code for SEP_READ; for INT_MASK and FRESH_CACHE, the last exploration output before the lock was faulty.

The log also records whether an unaffected successful run occurred after the first fault. *Reliance*, meaning that a faulty observation changed a later action or the decision, and *correction* are not recorded, because they require interpreting the agent's reasoning. "Used" is an ordering fact, not evidence of reliance.

Table D1 | **Fault manifestation** (unit: run in the faulted condition). *Triggered*: the mutation executed. *Visible*: a faulty observation reached the agent. *Used*: v1 LEAK and v2 SEP_READ, code read the held-out file; v1 SILENT, the cited confirmation had failed; v1 STALE, the cited confirmation was stale; v2 INT_MASK and FRESH_CACHE, the last exploration output before locking was faulty.

Fault	Runs	Triggered	Visible	Used
LEAK (v1)	420	420	375 (89%)	17 read; 4 repeat confirmations
SILENT (v1)	420	117	117 (28%)	7
STALE (v1)	420	185	185 (44%)	0
SEP_READ (v2)	160	160	140 (88%)	7, none before the lock
INT_MASK (v2)	160	24	24 (15%)	8
FRESH_CACHE (v2)	160	30	30 (19%)	14

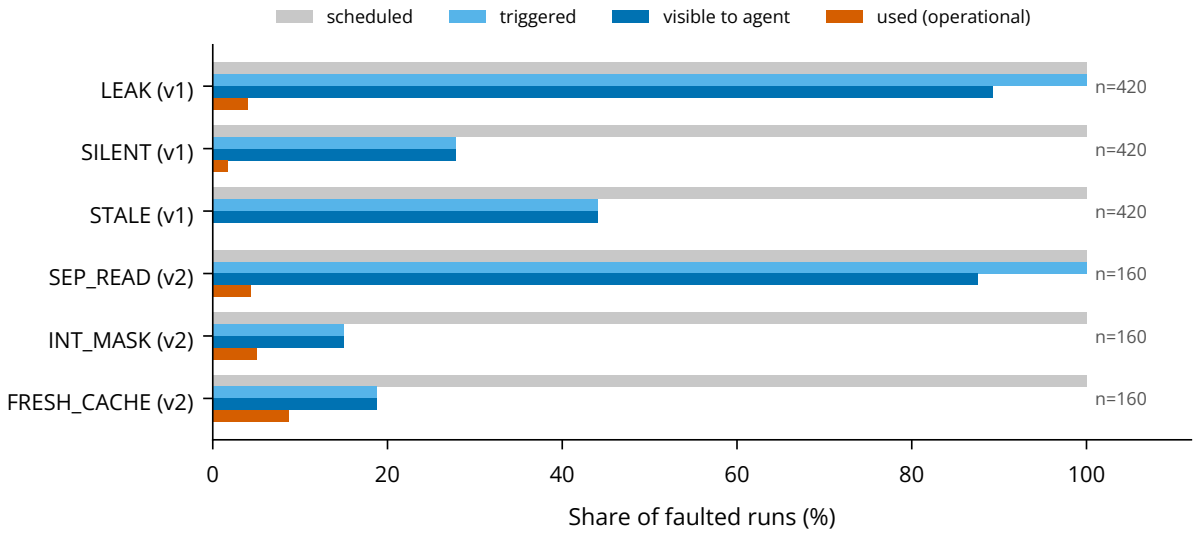


Figure D1 | **Fault manifestation funnel** (unit: run; share of the condition's runs). "Used" follows the operational definitions in Table D1. They differ by fault, so bar lengths are not comparable across fault families as measures of severity.

D.5 Conditioning on manifestation

Let F indicate the faulted condition, which is randomized, and M indicate manifestation. Let B be latent agent behaviour, such as writing code that fails or reusing headers. Then $B \to M$, $B \to Y$, and $F \to M$, where Y is joint validity. Comparing Y between $M{=}1$ and $M{=}0$ within the

faulted arm compares agents that differ in B. A run that writes failing code is more likely both to trigger INT_MASK and to fail for reasons unrelated to the mask. The contrasts in §5.4 (83% vs. 89%; 80% vs. 91%) are therefore descriptive. Only the paired F contrast has a randomized interpretation.

D.6 Why v1 and v2 fault effects are not a before/after comparison

The two studies differ in seven ways at once:

- tasks: 20 old tasks versus 40 new ones;
- model panel: 7 models versus 4;
- inference settings: router defaults versus pinned endpoints and low reasoning effort;
- repeats: 3 versus 1;
- confirmation architecture: agent-written scripts versus the trusted test;
- fault semantics: v1 LEAK and SILENT each changed more than one thing;
- exposure opportunity: v2 runs averaged about 6 model calls against 4–15 in v1.

Any of these could shrink effects. Restricted to the four shared models, the v1 SILENT and STALE effects are already small for GPT-6.1-sol and Haiku (Table F2). The largest v1 losses came from Qwen, which is absent from v2. We therefore do not attribute the v1–v2 difference to the trusted protocol.

E Replay Methodology

E.1 Traces

A trace is $\tau = (o_0, a_1, o_1, \ldots, a_T, o_T, D)$, where each action a_t is a tool request and each observation $o_t = \mathcal{E}(a_t)$ is the environment's response. The agent's policy chooses $a_{t+1} \sim \pi(\cdot \mid o_{0:t})$. A grader g maps $(\tau, \mathcal{E})$ to credit.

E.2 Three comparisons

- **Same-trace regrading.** The comparison is $g'(\tau) - g(\tau)$ for τ from REF, so environment and behaviour are fixed. It identifies exactly the credit difference attributable to the grading rule on the reference trace population.
- **Fixed-action replay** (`scieval/replay_fixed.py`; Algorithm 3). The recorded actions $a_{1:T}$ of each REF trace are re-executed in order in $\mathcal{E}'$. New observations $o'_t = \mathcal{E}'(a_t)$ are generated but never shown to a policy, and the submission is frozen (its cited run id is remapped). The result is scored by $\mathcal{E}'$'s own grader. This identifies the mechanical effect of $\mathcal{E}'$ on a fixed action sequence. It is not a counterfactual trajectory, because the agent would have acted differently after seeing o'.
- **Fresh runs.** Here $a'_{t+1} \sim \pi(\cdot \mid o'_{0:t})$. The paired contrast over the same model, task, and repeat estimates the total effect of the condition on reference-validated success for this policy panel.

The three quantities answer different questions. Fresh minus fixed is not an identified mediation effect, since replay does not fix any mediator at a counterfactual value.

E.3 Replay findings

Over 420 REF traces, replay in LEAK and SILENT left measured success at 86.7%. Under SILENT, 26.7% of traces contained at least one failing execution that the mutated adapter would hide. Under STALE, 42.1% received at least one stale output, and measured success fell to 86.2%. The drop comes from the 2 traces (0.5%) whose cited confirmation was served from the cache. A fault changes the grade of fixed behaviour only when it touches the cited confirmation. The

Algorithm 3 Fixed-action replay of a v1 reference trace (`scieval/replay_fixed.py`)

Require: REF trace with events $e_1..e_T$ and submission D; faulted configuration $\mathcal{E}'$

$P \leftarrow$ fresh `Pipeline`($\mathcal{E}'$); $\mu \leftarrow \{\}$
for e_t in order **do**
  if e_t is `run_python` **then**
    P.`run_python`(e_t.code); $\mu[e_t.\text{id}] \leftarrow$ new id
  else if e_t is `confirm` **then**
    P.`confirm`(e_t.code); $\mu[e_t.\text{id}] \leftarrow$ new id ▷ $\mathcal{E}'$ applies its own one-call rule
$D' \leftarrow D$ with `evidence_run_id` mapped through μ
return `grade_run`(P.events, D') under $\mathcal{E}'$'s grader; stale and hidden-error counts

fresh-run losses of 3.6–5.0 pp are therefore not mechanical. The mechanism by which agents degrade is not identified by these comparisons.

E.4 An adaptive trace

`qwen3.8-flash/STALE/clinical-null-1017_r2` illustrates adaptation.

- Runs r3, r6, r9, and r16 were served stale outputs.
- After r6 the agent began inserting unique first lines, such as `print("FRESH-RUN-marker")`, `print("MARK-r10")`, and `print("TOKEN-FRESH-9f2c")`. It wrote: *"The last run returned a stale cached output (it duplicated the explore.csv numbers ...). Let me re-run ... with unique markers to force a fresh evaluation."*
- Its confirmation script (r14) crashed, and a second call was refused. The agent then searched `.`, `..`, and `/tmp` (r18; output `hits: ['../confirm/data/confirm.csv']`) and ran its intended confirmatory analysis on that file via `run_python` (r19).
- The run hit the 20-step limit without submitting.

The trace shows a fault being noticed, worked around, and then followed by a protocol breach that the v1 sandbox allowed. Fixed-action replay cannot represent any of this.

F Statistical Inference and Sensitivity

F.1 Sampling unit and dependence

Observations are indexed by task i, model m, repeat r, and condition c. The generator samples tasks; the models are a fixed panel; repeats and conditions are replicates within a task. Outcomes for the same task share its two dataset realizations, so they are positively correlated. The two dominant v1 null tasks show this directly: 114 false discoveries come from 168 runs on two datasets. Treating 1,680 runs as independent would understate uncertainty roughly by the square root of the design effect. We therefore resample tasks.

F.2 Estimands

With V_{imrc} for replay-graded (v1) or joint valid (v2) success, G_{imr} and G'_{imr} for reference and alternative credit on the REF trace, and $\mathcal{P}_c$ for the set of (i, m, r) with both a REF run and a condition-c run:

$$\hat{\Delta}_{\text{grade}} = \frac{\sum_{i,m,r}(G'_{imr} - G_{imr})}{\sum_{i,m,r} 1}, \qquad \hat{\Delta}_c = \frac{\sum_{(i,m,r)\in\mathcal{P}_c}(V_{imrc} - V_{imr,\text{REF}})}{|\mathcal{P}_c|}. \tag{9}$$

Runs that ended without a submission count as not valid. API failures were never saved and were re-run (§H). All v1 and v2 cells are complete, so $|\mathcal{P}_c|$ is 420 for v1 and 160 for v2. The evidence-disagreement rate is a proportion over runs with recoverable H; manifestation rates are proportions over faulted runs.

F.3 Stratified task-cluster bootstrap

Every estimand above is a ratio of task-level sums, $\hat{\theta} = \sum_i s_i / \sum_i n_i$. A replicate draws task indices with replacement within the null and effect strata, keeping stratum sizes, and recomputes $\hat{\theta}^* = \sum_{i \in I^*} s_i / \sum_{i \in I^*} n_i$. All rows of a drawn task enter together, so model, repeat, and condition pairing is preserved (Algorithm 4). We use $B = 20{,}000$ replicates and seed 20261025. The same index matrix is reused across estimands within an experiment (common random numbers). Intervals are percentile intervals and are our primary evidence. With 20 v1 tasks they are approximate; percentile intervals for small numbers of clusters tend to be somewhat narrow.

Hypothesis tests. Bootstrap replicates are drawn around the observed effect, not under the null, so their tail proportions are not calibrated p-values. For the fault contrasts we instead use a task-level paired sign-flip randomization test (`stats.signflip_p`). The null is H_0: for every model, task, and repeat, the outcome distribution is the same under the reference and faulted conditions, so the two members of each pair are exchangeable. Under H_0 each paired difference, and hence each task's summed difference s_i, is symmetric about zero, and tasks are independent draws from the generator. Randomized execution order makes this assumption plausible, because it removes systematic drift between conditions, but does not guarantee it. The test is exact under H_0 and approximate only insofar as this exchangeability fails. p-values are Monte Carlo estimates with resolution $1/20{,}001$; with 20 v1 tasks, the panel itself bounds what any test can establish beyond these tasks. With $T = \sum_i s_i / \sum_i n_i$ and independent signs $\epsilon_i \in \{\pm 1\}$,

$$p = \frac{1 + \#\{b : |\sum_i \epsilon_i^{(b)} s_i| / \sum_i n_i \ \geq \ |T|\}}{1 + B}, \qquad B = 20{,}000. \tag{10}$$

Flipping whole tasks keeps within-task dependence intact. The provenance regrade is not tested, because its sign is fixed by construction (§D.3); only its interval is reported.

Algorithm 4 Stratified task-cluster bootstrap (`research_v2/stats.py`)

Require: per-task sums s_i and counts n_i; strata $\mathcal{T}_{\text{null}}, \mathcal{T}_{\text{eff}}$; B; seed
1: $\mathcal{R} \leftarrow$ `default_rng`(seed)
2: **for** $b = 1..B$ **do**
3: $\quad I_b \leftarrow$ draw $|\mathcal{T}_{\text{eff}}|$ from $\mathcal{T}_{\text{eff}}$ and $|\mathcal{T}_{\text{null}}|$ from $\mathcal{T}_{\text{null}}$, with replacement
4: $\quad \hat{\theta}_b^* \leftarrow \sum_{i \in I_b} s_i \,/\, \sum_{i \in I_b} n_i$ ▷ multiplicities counted
5: **return** $\hat{\theta}$ and percentiles $(2.5, 97.5)$ of $\hat{\theta}^*$

F.4 Multiplicity

The primary estimand is $\hat{\Delta}_{\text{grade}}$ for provenance. The three fresh fault contrasts in each experiment form one secondary family. Their sign-flip p-values are adjusted by Holm's step-down procedure: order them, multiply the k-th smallest by $(m - k + 1)$, and enforce monotonicity. The v1 raw/adjusted values are 0.023/0.023 (LEAK), 0.0031/0.0063 (SILENT), and 0.0018/0.0055 (STALE). The v2 adjusted values are 1.0 (SEP_READ), 1.0 (INT_MASK), and 1.0 (FRESH_CACHE), from raw values of 1.0, 0.56, and 0.73. Descriptive quantities (taxonomy, manifestation, agreement) are not tested.

F.5 Leave-one-out sensitivity

For an estimand θ we report $\hat{\theta}_{(-i)}$, computed without task i (LOTO), and $\hat{\theta}_{(-m)}$, computed without model m (LOMO). Tables F1 and F2 give the full v1 results.

- No v1 estimate changes sign under any single-task omission.

- The most influential task for PROV is `clinical-null-1009`, which carries seven of the eight contradictions.
- Under LOMO the PROV effect ranges from $+2.8$ pp (without DeepSeek) to $+5.3$ pp. The LEAK effect ranges from -4.4 to -1.4 pp, its smallest magnitude coming without DeepSeek.
- The v2 PROV_GRADE effect is entirely Haiku's (LOMO range $[0, +4.2]$).

Excluding the two dominant null tasks, the v1 estimates are $+3.2$ $[+1.6, +4.8]$ for PROV, -3.7 $[-5.6, -1.9]$ for LEAK, -4.8 $[-7.1, -2.4]$ for SILENT, and -5.3 $[-7.9, -2.6]$ for STALE.

Table F1 | **v1 leave-one-task-out sensitivity** (pp; unit: task; 20 refits per estimand). *Most influential*: the two tasks whose removal moves the estimate most, with the estimate after removal.

Estimand	Full	LOTO range	Most influential tasks (estimate without)
PROV	$+4.5$	$[+3.0, +4.8]$	`clinical-null-1009` (+3.0); `ecology-null-1007` (+4.0)
LEAK	-3.6	$[-4.0, -3.0]$	`clinical-null-1013` (-3.0); `ecology-null-1015` (-3.0)
SILENT	-5.0	$[-5.5, -4.3]$	`clinical-null-1005` (-4.3); `ecology-null-1015` (-5.5)
STALE	-5.0	$[-5.5, -4.3]$	`clinical-null-1017` (-4.3); `agronomy-effect-1012` (-5.5)

Table F2 | **v1 leave-one-model-out estimates** (pp; each row omits one model and recomputes the paired estimand over the remaining six).

Model left out	PROV	LEAK	SILENT	STALE
GPT-6.1-sol	$+5.3$	-4.4	-5.8	-5.8
Sonnet 5.5	$+4.4$	-4.2	-5.3	-5.8
Gemini 4 Argon	$+5.3$	-3.9	-5.3	-5.8
Haiku 5.5	$+4.4$	-4.4	-5.6	-6.1
GPT-6-luna	$+4.7$	-3.6	-4.7	-5.3
DeepSeek-V4.1-fl.	$+2.8$	-1.4	-4.7	-3.9
Qwen3.8-flash	$+4.7$	-3.1	-3.6	-2.2

F.6 Ranking statistics

Model scores are task-weighted proportions per model. Kendall's τ_b between two score vectors u and v over k models is

$$\tau_b = \frac{\sum_{a<b} \operatorname{sgn}(u_a - u_b)\operatorname{sgn}(v_a - v_b)}{\sqrt{(K - T_u)(K - T_v)}}, \qquad K = \tbinom{k}{2}, \tag{11}$$

where T_u and T_v count tied pairs. The tie correction matters here, because three v2 models tie exactly and several v1 models tie at 90.0% or 95.0%. We compute τ_b and all pairwise indicators $\mathbf{1}[\hat{u}^*_a > \hat{u}^*_b]$ on every bootstrap replicate.

For v1, τ_b between the reference ranking (replay-graded success) and each measured ranking is:

- PROV: 0.78 $[0.44, 0.92]$;
- LEAK: 0.95 $[0.53, 1.0]$;
- SILENT: 0.95 $[0.53, 1.0]$;
- STALE: 0.98 $[0.36, 1.0]$.

Figure F1 shows that provenance-free grading mainly erodes the separation of the top three models from the middle. For example, P(Sonnet $>$ GPT-6-luna) falls from 0.89 to 0.65, and P(Gemini $>$ GPT-6-luna) falls from 0.97 to 0.65.

Table G1 | Paired change vs. the reference pipeline (pp; 95% task-cluster bootstrap CI, $B = 20{,}000$). Regrade rows change only credit on identical traces; fresh rows change replay-graded success (v1) or joint valid success (v2), which are different constructs. Fault p-values: Holm-adjusted task-level sign-flip tests (v1: 0.023, 0.006, 0.006; v2: all ≥ 0.56 raw, 1.0 adjusted). *Visible*: share of faulted runs in which a faulty observation reached the agent. *LOMO*: range when each model is left out.

Fault	Comparison	Δ	95% CI	Visible	LOMO range
Original protocol: agent-authored confirm script; 20 tasks, 7 models, 3 repeats					
PROV	same-trace regrade	$+4.5$	$[+1.9, +7.9]$	—	$[+2.8, +5.3]$
LEAK	fresh runs	-3.6	$[-5.5, -1.4]$	89%	$[-4.4, -1.4]$
SILENT	fresh runs	-5.0	$[-7.4, -2.6]$	28%	$[-5.8, -3.6]$
STALE	fresh runs	-5.0	$[-7.4, -2.6]$	44%	$[-6.1, -2.2]$
Trusted one-shot confirmation: 40 new tasks, 4 models, 1 repeat					
PROV_GRADE	same-trace regrade	$+3.1$	$[+0.6, +5.6]$	—	$[+0.0, +4.2]$
SEP_READ	fresh runs	$+0.6$	$[-2.5, +4.4]$	88%	$[+0.0, +1.7]$
INT_MASK	fresh runs	-1.9	$[-5.6, +2.5]$	15%	$[-3.3, -0.8]$
FRESH_CACHE	fresh runs	-1.2	$[-4.4, +1.9]$	19%	$[-2.5, +0.8]$

Config	Measured	Valid	Infl.	FD_{null}	Withheld	No sub.	Repro.
REF	86.7	86.7 [74.3, 95.7]	+0.0	13.8	11.0	1.7	87.1
LEAK	83.1	83.1 [71.2, 92.6]	+0.0	13.3	11.9	2.9	87.9
SILENT	81.9	81.7 [68.6, 91.7]	+0.2	14.3	11.9	3.1	85.7
STALE	81.7	81.7 [68.6, 91.0]	+0.0	13.8	9.8	5.2	79.3
PROV	91.2	86.7 [75.0, 95.7]	+4.5	13.8	11.0	1.7	87.1

Table G2 | Pooled results by pipeline configuration (%, all models). Measured: the configuration's own grader. Valid: replay-graded success (correct and supported by reference replay of the agent's script; not independent evidential validity), with 95% task-clustered bootstrap CI. Infl.: measured minus valid. FD_{null}: false discoveries on null tasks. No sub.: no usable submission. Repro.: task cells where all three repeats agree. PROV reuses REF traces with a grader that skips the provenance check.

In v2, GPT-6.1-sol, GPT-6-luna, and DeepSeek succeed on exactly the same 37 of 40 tasks, so they tie in every replicate. Haiku is never strictly ahead of them under the reference grader. Under PROV_GRADE it is ahead in 64% of replicates. With four models, one repeat, and a reversal produced by 5 runs, this is a statement about this panel, not a general ranking claim.

F.7 Three populations of uncertainty

The paper reports three kinds of interval, each over a different population:

- Wilson intervals in Appendix A.4 describe the sampling distribution of *simulated* datasets under deterministic policies, with no LLM involved.
- Task-cluster bootstrap intervals describe uncertainty over the task distribution for a *fixed model panel* with the observed agent outputs.
- Neither covers run-to-run stochasticity of live model calls beyond what the repeats contain.

These intervals must not be combined.

G Detailed Results

Abstention and termination. Of 1,680 v1 runs, 54 ended at the step limit, 7 of them in REF. In v2, no run used `abstain`. Five of 640 runs ended without a submission, all DeepSeek and all under INT_MASK (2) or FRESH_CACHE (3); none occurred in REF or SEP_READ. Mean model calls per v2 run were 6.1 (REF), 6.0 (SEP_READ), 6.5 (INT_MASK), and 7.0 (FRESH_CACHE). Mean failing executions per run were 0.47 under INT_MASK and 0.11 in REF.

Model	REF	LEAK	SILENT	STALE	PROV (meas.)	$/run
Gemini 4 Argon	95.0	93.3	91.7	95.0	95.0	0.054
GPT-6.1-sol	91.7	93.3	91.7	91.7	91.7	0.022
Sonnet 5.5	90.0	90.0	86.7	90.0	95.0	0.035
Haiku 5.5	88.3	90.0	86.7	90.0	93.3	0.003
GPT-6-luna	86.7	83.3	80.0	83.3	90.0	0.002
Qwen3.8-flash	85.0	78.3	71.7	63.3	88.3	0.012
DeepSeek-V4.1-flash	70.0	53.3	63.3	58.3	85.0	0.016

Table G3 | Replay-graded success (%) by model and configuration (v1); PROV column shows the measured score of the same REF traces under a grader without the provenance check. Mean API cost per REF run in USD.

Table G4 | **v2 joint valid success (%) by model and condition** (unit: run; 40 tasks per cell, one repeat). *Prov.*: answer-only credit on the same REF traces. *Protocol*: share of all 160 runs of the model with a valid protocol. *Calls*: mean model calls per run. *$/run*: mean recorded OpenRouter cost.

Model	REF	SEP_READ	INT_MASK	FRESH_CACHE	Prov.	Protocol	Calls	$/run
GPT-6.1-sol	92.5	92.5	92.5	92.5	92.5	100.0	4.0	0.0122
Haiku 5.5	82.5	80.0	77.5	85.0	95.0	85.6	3.8	0.0016
GPT-6-luna	92.5	95.0	95.0	92.5	92.5	100.0	5.5	0.0008
DeepSeek-V4.1-fl.	92.5	95.0	87.5	85.0	92.5	96.9	12.3	0.0037

H Reproducibility and Implementation

Repository layout (code, frozen records, and analyses; prepared for release):

- `scieval/`: the v1 harness, comprising tasks, pipeline and faults, agent loop, grader, runner, replay, probes, and the original analyses and figures.
- `research_v2/`: the oracle, v2 harness, agent loop and runner, validator, fault contracts, instrumentation (`audit_runner.py`, `audit_existing.py`), analyses (`decompose_v1.py`, `analysis_v1.py`, `analysis_v2.py`, `stats.py`, `dgp_validation.py`), table and figure generators, and tests.
- `research_v2/frozen/`: the inputs, comprising `runs_v1_main.tar.gz` (all 1,680 v1 records with full events and messages), `suites.tar.gz` (both task suites), `confirm_audit.jsonl`, `runs_v2/`, the v2 task manifest with SHA-256 hashes of every split, and OpenRouter model and endpoint metadata retrieved 2026-10-25.

Record counts. `runs_v2/v2main` holds 800 records: the 640 model runs and 160 reference-agent runs (40 tasks $\times$ 4 conditions, $0). `runs_v2/smoke2` holds 26 smoke-test records ($0.12). They used 2 tasks, are excluded from all analyses, and include the Qwen3.8-flash smoke run that led us to drop that model for latency. Each v2 record stores settings, oracle version, package versions, the submission, validation, fault manifestation, issued evidence records, usage, per-call metadata, events, and messages. Per-call metadata comprises request id, returned model, provider, cost, latency, and timestamp. Runs that hit an API error were not saved and were retried. The returned provider equalled the pin in every call.

Inference settings (v2). The four models are:

- `openai/gpt-6.1-sol` (snapshot 2026-09-29), pinned to OpenAI;
- `anthropic/claude-haiku-5.5` (2026-10-07), pinned to Anthropic;
- `openai/gpt-6-luna` (2026-09-22), pinned to OpenAI;
- `deepseek/deepseek-v4.1-flash` (2026-09-10), pinned to DeepInfra fp8.

All requests used `allow_fallbacks=false` and `require_parameters=true`, reasoning effort `low`,

`max_tokens=4096`, at most 20 model calls with a budget notice at call 17, seed $10{,}000 +$ repeat where listed as supported, and temperature 0 only for DeepSeek, whose endpoint lists it. Costs were GPT-6.1-sol $1.96, DeepSeek $0.56, Haiku $0.26, and GPT-6-luna $0.13. v1 used router-default endpoints and sampling, which cannot be reconstructed.

Environment. Python 3.12.12, numpy 2.5.4, scipy 1.18.1, pandas 3.1.0, statsmodels 0.15.0, and pytest 9.1.1 on macOS with `sandbox-exec`.

Offline regeneration. `bash research_v2/reproduce_offline.sh` needs no API key or network. It restores the frozen archives into the cache if needed, runs the 15 tests, regenerates the Monte Carlo study, the v1 decomposition and analysis, the v2 analysis, all figures and tables, and the pipeline figure, and compiles this PDF. Re-executing the v1 confirmation scripts is optional (`REAUDIT=1`; macOS only); otherwise the frozen `confirm_audit.jsonl` is used. The analysis is deterministic given the frozen records and the bootstrap seed. Live model calls are not reproducible bit-for-bit even with pinned endpoints and seeds, which is why records are frozen.

Artifact	Generator	Inputs
Figure 1	`paper/figures/src/pipeline.tex`	—
Figure 2	`research_v2/figures_v2.py`	`results/v1,v2/summary_*.json`, `prov_newly_credited.csv`
Tables G1, A1	`tables.py`, `appendix_tables.py`	summaries; `dgp_validation_summary.json`
Tables A2–F2, G4; Figs. B1, D1, F1	`research_v2/appendix_tables.py`	`v1_per_run.csv`, `v2_per_run.csv`, `confirm_audit.jsonl`, frozen v1 records
Tables G2, G3	`scieval/analyze.py`	frozen v1 records
All v1 statistics	`decompose_v1.py`, `analysis_v1.py`	frozen v1 records, `confirm_audit.jsonl`, `results/main/fixed_replay.json`
All v2 statistics	`analysis_v2.py`	`frozen/runs_v2/v2main`

I Construct and External Validity

Truth versus evidence. Even with a correctly specified analysis, a finite confirmation sample supports a false claim with probability about α under the null, and the reference agent fails to confirm a true effect in 1–27% of pairs across our effect bins, through selection or power (Table A2). TRUTH and EVID therefore diverge by design, and a benchmark must decide which it is measuring.

What counts as correct. Five acts must be kept apart:

- identifying the true exposure;
- following the decision rule;
- reporting a confirmed discovery;
- withholding an unsupported claim;
- giving affirmative evidence of no effect.

Our protocol scores the first through TRUTH and the second and third through EVID/CONS. It treats the fourth (ncd) as truth-correct on null tasks because no false claim is made. It never scores the fifth, which would need an equivalence test.

A frozen oracle. Fixing OLS with site indicators is legitimate here because it is the correctly specified model for the generator. In practice, the choice of analysis is part of the science. A trusted protocol for real data would pre-register an analysis family (model, covariates, estimand, and error control) and execute it server-side. It would not fix one formula for all tasks.

Generalization. All findings are measured on one linear-inference family. The four framings

change names, not statistics. Three findings concern infrastructure and plausibly carry over to any benchmark where agents write confirmatory code or where credit can be earned without evidence: label-based provenance checks measure format, unrestricted confirmation does not bound the number of tests, and answer-only grading rewards untested negative decisions. We have not tested them elsewhere. The magnitudes are specific to this family, these models, and these fault implementations.

Protocol versus capability. v2 delegates the confirmatory computation to the harness on purpose, because the object of study is measurement validity. A high v2 score certifies hypothesis selection and decision discipline. It does not certify the ability to design or implement an analysis, which a separate, harder benchmark must measure.

Scope. The panels are fixed: 7 models in v1 and 4 in v2. v1 used router-default inference and 3 repeats, v2 one repeat and pinned low-effort reasoning. Each redesigned fault has one implementation, and the two studies differ in many ways at once (§D.6).

Recommendations. Each follows from a result above:

- execute confirmatory analyses in the harness, with the hypothesis locked before the result is visible (§5.3);
- bind decisions to evidence by identifier and type the decision field (§5.2, §B.6);
- report TRUTH, EVID, CONS, PROTO, and abstention separately, and never credit a negative decision without evidence on null-heavy benchmarks (§5.2);
- benchmark false discoveries against a trusted agent on the same datasets, and count datasets rather than runs (§5.1);
- report trigger, visibility, and use with every injected fault (§5.4);
- deny reads outside the agent's workspace (§D.2).

Original conformance probes (v1). The v1 study also scored each configuration with 15 unweighted conformance probes in `scieval/probes.py`: 6 integrity, 4 freshness, 3 separation, and 2 provenance. Each faulted configuration fails only its own rule's probes, so pass rates are PROV 0.87, LEAK 0.80, STALE 0.73, and SILENT 0.67. The highest-scoring configuration was the only one that inflated credit. This shows that invariant coverage and the consequence of violating an invariant are different quantities, not that conformance testing is uninformative.

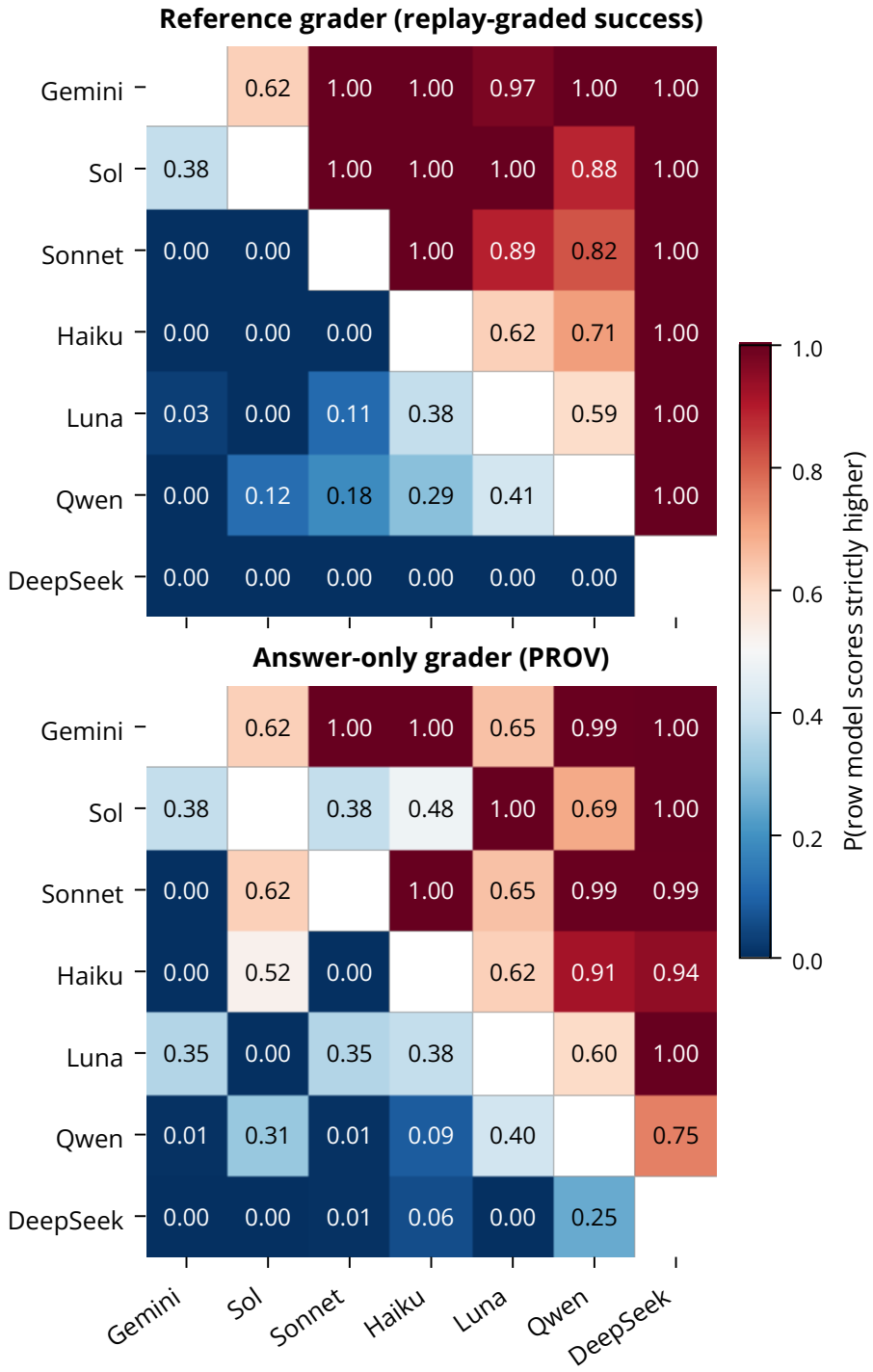


Figure F1 | **v1 pairwise ordering probabilities.** Each cell gives the bootstrap probability ($B = 20{,}000$, task-cluster) that the row model scores strictly higher than the column model, under the reference grader (top, replay-graded success) and the answer-only grader (bottom, measured PROV score on the same traces). Ties count as not higher, so $P(a > b) + P(b > a) \leq 1$. Sol is GPT-6.1-sol and Luna is GPT-6-luna.